Security & Compliance

Polycarpic — a virtual Single Family Office

Owner: `seceng` agent

Status: Draft v1.2.3. A-16 to A-27 are folded in ARCH v1.5. SecEng rulings on ARCH OQ-30, OQ-31, OQ-32 and OQ-34, and on the email-anchored first enrolment, are in §12.1. Pending: A-28 to A-33 (§12.2), and the PRD/DESIGN copy of the amended consent text (C-25), of the own-Person exemption and of the email-anchored set-up (§12.3).

Traces to: PRD v1.4, ARCH v1.5 draft, DESIGN v0.16, DECISIONS through 2026-11-08

Last updated: 2026-11-08

Changelog

- **v1.2.3 (2026-11-08).**
  - BetterAuth claims measured against better-auth 1.7.18 and @better-auth/passkey 1.7.18 on PostgreSQL 14 (POLY-62 spike); none is marked unmeasured any more (§4.1 "BetterAuth, measured").
  - R-12 confirmed: session tokens are stored in the clear. The impact is narrowed, because the cookie is signed with the auth secret.
  - Corrected: backup codes are encrypted, not hashed (§4.1, §5). `encryptOAuthTokens` is turned on, and the IdP ID token is dropped (§5, A-33).
  - Rotating the BetterAuth secret goes through its versioned `secrets`, because the secret also encrypts TOTP seeds and backup codes (§6.1).
  - The passkey endpoints are `generate-register-options` and `verify-registration`; `/passkey/add-passkey` does not exist in 1.7.18 (§4.1, C-03, T-83).
  - The database guard needs a change. BetterAuth's writes run outside our transaction, and the first TOTP verify replaces the session before it marks TOTP verified. So the guard moves, and BetterAuth's pool is routed through `withTenant` (§12.2: A-29 amended, new A-31 to A-33).
- **v1.2.2 (2026-11-08).**
  - The first second-step factor is enrolled only after a code sent to the account email is confirmed, in addition to the password (Principal, 2026-11-08). A stolen password alone can no longer enrol the thief's own factor at the OQ-34 gate (§4.1, C-03, T-82, T-83).
  - An account with no second step is deleted with the password plus the same email confirmation, with no factor set-up (§4.6, C-52, T-84).
  - Ruled: completing a first enrolment counts as a fresh step-up in that session, so the next Confirm it's you passes without retyping (§4.1).
  - BetterAuth has no email-verified enrolment step; the app wraps the enrolment endpoints (§4.1; measured in v1.2.3).
  - §12.2: new A-29, A-30. §12.3: new PRD and DESIGN asks.
- **v1.2.1 (2026-11-08).**
  - OQ-34 (Principal, 2026-11-08): the owner row on a user's own Person is exempt from the second step while that Person is unshared; the first real action forces enrolment (§4.1, C-03, R-7).

    - The held role uses ARCH §3.1c's mechanism, `membership.pending_role`, which meets A-25; its edges are in §4.1.
    - Step-up adds removing a second-step method, the co-owner request steps, editing the cap table, cancelling a held raise, and transferring or stepping down from the founder role (§4.1).
    - §12.2: A-23 to A-27 folded in ARCH v1.5; new A-28.
- **v1.2 (2026-11-08).**
    - OQ-30: `poly_journal` confirmed with narrowed grants; the counter row is created by its trigger, and a line can no longer be added once its entry is chained (C-24, T-80).
    - OQ-31: direct invitations are revoked at the 1 → 2 owner transition, under a lock (§4.2.2, C-28).
    - OQ-32: C-76's startup check tests the zone, not the host.
    - Consent text (C-25): fixed as three sentences, with the first amended to name every parent above.
    - Sign-in providers are Google, Apple and GitHub (Principal, 2026-11-08); C-05 pins how each asserts a verified email.
    - A raised role is not effective until the second step is enrolled (§4.1, C-03, T-81).
    - §12.2: A-16 to A-22 folded; new A-23 to A-27. §12.3: the ux-designer's asks are closed.
- **v1.1 (2026-11-07).**
    - Nesting, roll-up and cap table: §3.8, C-19 amended, new C-25 and C-27, and new roles `poly_graph` and `poly_stake`.
    - Account deletion in-flow: §4.6 and C-52.
    - The purge delay is removed (OQ-23, R-16).
    - Staging keyring and Plaid Sandbox (C-77).
    - Local-network box posture (C-76).
    - Vercel is now the tertiary target.
    - New risks R-15 to R-21; every §11 row is approved.
    - Principal rulings P1–P9, P5b, R-19 and R-20:
        - MFA for owner and manager;
        - any-range roll-up, with consent disclosure;
        - the hash chain in POLY-M2;
        - the ancestor-invite notice;
        - co-owner approval, with a founding owner;

    - no sign-up switch;
    - local-box TLS, TPM unlock and backups.
- **v1 (2026-09-28).** Reviewed rewrite that supersedes 8963efb.

CONTENTS

1. Compliance Posture
2. Trust Boundaries
3. Threat Model (STRIDE)
4. Authentication & Authorization
5. Data Classification & Handling
6. Secret Management
7. Controls Catalog
8. CI/CD Security
9. Logging, Monitoring, Detection
10. Incident Response
11. Open Risks (Accepted)
12. Rulings on ARCH Open Questions

1. Compliance Posture

Source of truth is PRD §6 (Non-Functional Requirements) and the Principal's audience decision of 2026-09-27.

Polycarpic is software a person installs and runs for their own household. The Principal will never operate it as a service for others. That is a standing non-goal (PRD §3), not a v1 deferral. Nobody is anybody's "customer", so no formal compliance regime attaches.

That does not lower the bar. The Principal made the PRD §6 security items hard requirements, and this document treats them that way. Every control here answers a threat in §3, not an auditor. The yardstick is **OWASP ASVS 4.0 Level 2**, used as a checklist, not as a certification target.

Regime	In scope?	Why	Target evidence
GLBA Safeguards Rule	No	Applies to financial institutions that serve customers, and a self-hosted personal tool has none. It would likely attach if the hosting non-goal were reversed (R-6).	—
PCI-DSS	No	We never store, process or transmit card numbers. Aggregators expose transactions, not PANs. Adapters must not persist a PAN even if a provider payload carries one (C-34).	C-34 adapter fixture test
GDPR / CCPA	No (voluntary alignment)	Processing by a natural person in a purely personal or household activity falls under the GDPR Art. 2(2)(c) household exemption. CCPA thresholds are never met. We adopt the data-rights practices anyway (PRD §6): export, account deletion and entity purge, all scoped to the immutable ledger.	C-50, C-51, C-52
SOC 2	No	No service organization and no customers.	—
HIPAA	No	No health data by design. Payee strings can still reveal medical providers, which is one reason memo and payee text is Tier 3 (§5).	—
Aggregator terms (Plaid, SimpleFIN)	Yes, per operator	Each self-hoster obtains their own provider keys and accepts that provider's terms. Plaid production access comes with a security questionnaire. The controls here are written so an operator can answer it truthfully.	§6, C-30 to C-35

Regime	In scope?	Why	Target evidence
Tax record retention (IRS guidance, 3–7 years)	Advisory	Not a regime we enforce. The purge flow must warn the owner about it (PRD §5).	C-50 warning copy

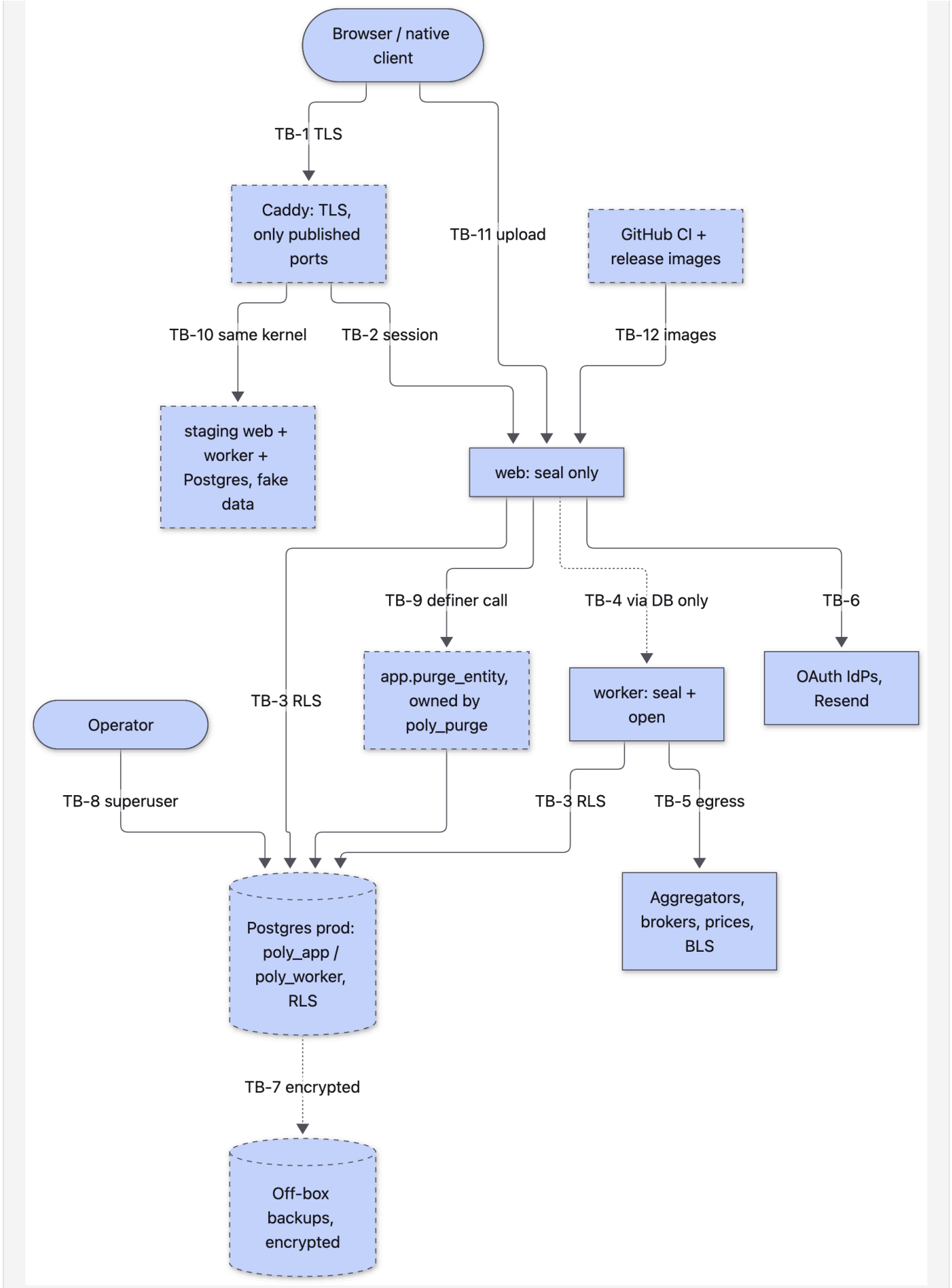
2. Trust Boundaries

ARCH §7 cites TB-1 to TB-10, so those numbers stay stable. The OQ-1 revision (§6.3) redefines TB-4. TB-11 and TB-12 are new. Each boundary becomes a threat entry in §3.

#	From	To	Boundary type	Notes
TB-1	Internet	Caddy	Network, TLS	TLS 1.2+ with HSTS. Caddy is the only container that publishes ports (80 and 443). *Local-network box (ARCH §7.8):* the "From" side is the LAN only. Nothing on the internet reaches Caddy, and the posture is C-76.
TB-2	Browser, native and MCP clients	SvelteKit `web`	Authentication	A session cookie or bearer token goes through `getSession()` and becomes a branded `SessionUserId`. Anything else is anonymous. CSRF and origin checks sit here.
TB-3	`web` / `worker`	Postgres	DB role + RLS	The primary authorization boundary. Apps connect as `poly_app` or `poly_worker`, never as the owner. Tenant context is transaction-local (§4.3).
TB-4	`web`	`worker` (the credential-opening capability)	Process + key separation	`web` can seal credentials but cannot open them; only `worker` holds the private key (§6.3). The two share only the database. *Revised:* 8963efb had a symmetric master key mounted into the app.
TB-5	`worker`	Aggregators, brokers, price and CPI providers	Egress, third-party trust	Outbound only. Credentials are opened just in time. Every response is untrusted input: payee and memo text can be written by anyone who sends the user money.
TB-6	`web`	OAuth IdPs, Resend	Third-party identit	The IdP asserts identity. Resend sees verification and magic-link URLs in transit.

#	From	To	Boundary type	Notes
			y and email	
TB-7	Box	Off-box backup store	Custody	Backups are encrypted before egress, with a key separate from the credential key (§6.6).
TB-8	Operator (SSH, `docker compose exec`)	Box, Postgres superuser	Administrative	Bypasses RLS and triggers by definition (R-1). C-24 detects what it does; it cannot prevent it.
TB-9	Ordinary write path	Purge path	Privilege (DB role)	The only path that deletes posted rows. It runs inside a function owned by the NOLOGIN `poly_purge` role (§4.5).
TB-10	Staging compose project	Prod compose project	Container isolation, shared kernel	Staging auto-deploys every merge to `main` and holds no real books. Its data is the fake source plus Plaid Sandbox and SimpleFIN demo data (Principal, 2026-11-06). It has separate networks, secrets and database. *Revised in v1.1:* staging has its own keyring, which seals sandbox tokens only, and it never holds prod's keyring or the production Plaid secret (C-77, R-4).
TB-14	Parent member (`poly_app`)	Aggregates of entities beneath the parent	Privilege (definer role)	The only path by which structure grants anything. Allowlisted definers owned by `poly_rollup` return fixed aggregate shapes, never rows. Their reads are confined by role-targeted policies to the caller's descendants. The cap-table and allocated-income definers (`poly_stake`) and the graph triggers

#	From	To	Boundary type	Notes
				(`poly_graph`) sit on the same boundary (§3.8, C-25, C-27).
TB-11	User upload (CSV / OFX)	`web` parser	Untrusted file	The credential-free import path, and a parser attack surface (C-40).
TB-12	GitHub (CI, Actions, Renovate, registry)	Release images, prod deploy	Supply chain	Whoever can publish an image that prod runs owns the box (§8).
TB-13	Operator (SSH tunnel)	Admin view listener, bound to loopback and never routed by Caddy	Administrative, out-of-band	*Placeholder; defined under POLY-81.* The design options are ARCH OQ-15: (a) a loopback route group inside `web`, or (b) a separate admin service with a read-only status DB role. The threats and controls are written when POLY-81 is ruled.


Browser / native client
TB-1 TLS
Caddy: TLS, only published ports
TB-11 upload
GitHub CI + release images
TB-10 same kernel
TB-2 session
TB-12 images
staging web + worker + Postgres, fake data
web: seal only
TB-9 definer call
TB-4 via DB only
TB-6
TB-3 RLS
app.purge_entity, owned by poly_purge
worker: seal + open
OAuth IdPs, Resend
Operator
TB-8 superuser
TB-3 RLS
TB-5 egress
Postgres prod: poly_app / poly_worker, RLS
Aggregators, brokers, prices, BLS
TB-7 encrypted
Off-box backups, encrypted

3. Threat Model (STRIDE)

One table per component group. Only threats with non-trivial likelihood × impact are listed. L = likelihood, I = impact (H/M/L). Control IDs refer to §7.

Attacker model.

- (a) An internet attacker who can reach Caddy.
- (b) A curious or malicious household member with a legitimate viewer or manager grant.
- (c) A counterparty who controls text that lands in our data. Anyone who sends the user money writes a payee or memo string.
- (d) A compromised dependency or CI path.
- (e) A thief with a backup or a disk.
- (f) An attacker with code execution in `web`. This one is in scope, and the design limits what they get.

The operator's own superuser is not modelled as an attacker (R-1).

3.1 TB-1 / TB-2 — Edge & web app (Caddy, SvelteKit, `packages/auth`)

ID	Threat	STRIDE	L	I	Control(s)
T-01	A stranger signs up on a public instance and gets an account on someone's household box.	S/E	H	M	C-01
T-02	Credential stuffing, spraying or brute force against password login.	S	M	H	C-03, C-04
T-03	OAuth account takeover: an IdP identity with an unverified email auto-links to an existing user.	S	L	H	C-05
T-04	Session cookie theft through XSS, subdomain cookie injection or the network.	S	M	H	C-02, C-07, C-41

ID	Threat	STRIDE	L	I	Control(s)
T-05	CSRF on state-changing form actions or `/api/v1`.	T/E	M	H	C-06
T-06	Stored XSS through provider payee or memo strings, or through entity or report names set by a co-member.	T/E	M	H	C-41, C-07
T-07	Account enumeration through login, reset or signup responses.	I	M	L	C-04
T-08	A stolen session is used for an irreversible or exfiltration-shaped action: purge, export, provider link, membership grant.	S/E	L	H	C-03 (step-up), C-50, C-52. A purge runs with no delay (R-16), so the controls are step-up, the typed name, a fresh export and an immediate email to every member.
T-09	Floods against expensive endpoints: reports, export, upload.	D	L	M	C-04, C-40, C-42
T-82	A password thief signs in to an account with no second step, on their own device, and at the OQ-34 gate sets up a factor they hold. The system has no earlier factor to compare against, so it would accept the first one offered. The thief then holds a "verified" second step and passes every later step-up: export, a bank link, sharing, a role raise.	S/E	M	H	C-03 (the first factor is enrolled only after an emailed code is confirmed, bound to the session). Residual: a thief who holds the password *and* the email account. Email is already the recovery root, since magic-link sign-in, invitations (C-01) and the new-device notice (§9) all go to it, so this adds no new root.

ID	Threat	STRIDE	L	I	Control(s)
T-83	The email anchor is bypassed: BetterAuth's `/two-factor/enable`, `/two-factor/verify-totp`, `/passkey/generate-register-options` or `/passkey/verify-registration` called directly; its `otp` method (email as the factor itself) enabled; the code guessed, replayed or carried into another session; or the code sent to a new address in the middle of an email change.	S / E	M	H	C-03 (endpoint wrapper, the database guard of A-29, single-use hashed code, attempt cap, session binding, the code goes only to the current verified address), C-04

3.2 TB-3 — Database & tenancy (`packages/db`, `packages/tenancy`, RLS)

ID	Threat	STRIDE	L	I	Control(s)
T-10	IDOR: a route accepts an `entityId` the caller has no membership on.	I / T	H	H	C-13, C-14, C-17
T-11	A pooled connection carries tenant A's context into tenant B's request.	I	M	H	C-11, C-12
T-12	A new table ships without RLS: staging, draft, lot or some future table.	I	H	H	C-13 (catalog-derived battery)
T-13	RLS is bypassed through ownership: a view runs as its owner, a materialized view has no	I / E	M	H	C-15, C-16

ID	Threat	STRIDE	L	I	Control(s)
	RLS, or a SECURITY DEFINER function is not on the allowlist.				
T-14	A write policy without `WITH CHECK`, or an updatable `entity_id`, lets a row be inserted into or moved to another entity.	T / E	M	H	C-14
T-15	A manager grants roles, or the last owner is removed.	E	M	H	C-20
T-16	A user attaches their entity under someone else's Household, injecting rows into the victim's roll-up reports.	T / S	L	M	C-19
T-17	Existence oracle: FK checks bypass RLS, so sequential IDs let an attacker probe for other tenants' rows.	I	M	L	C-18
T-18	SQL injection in `poly_app` sets `app.user_id` to anyone. The GUC is asserted by the app, not authenticated by the database.	S / I / E	L	H	C-44 prevents it; RLS does not (R-2)
T-19	Cross-tenant metadata read through user-defined entity types. This is by design (kickoff §2.4).	I	H	L	R-9; the C-14 view exposes the label only
T-26	On a multi-owner entity (co-trustees, partners), one owner acts alone. They admit a member or raise a role, remove a co-	E / T	M	H	C-28 (every other owner approves, enforced in the database; direct invitations revoked at the 1 → 2 transition), C-03 step-up. The

ID	Threat	STRIDE	L	I	Control(s)
	owner, or rewrite the cap table, possibly as a 0-unit bookkeeper owner or from a hijacked owner session.				founder's force path is narrow: co-owner removal or demotion only, after a decline or expiry, with notice. R-19.
T-81	A member raised to manager or owner gets the new role's power before enrolling a second step. A phished password then holds owner, or a pending owner counts toward the approver set or the 1 → 2 transition.	E	M	H	C-03 (the raise held in `pending_role` with `role` unchanged until enrolment, fail-closed), C-28 (direct-path held raises cancelled at 1 → 2)
T-20	Membership-table policies recurse through the authz functions, or FORCE RLS blinds those functions, so policies fail open or error.	E / D	M	H	C-15 (functions owned by `poly_authz`)

3.3 Ledger integrity — tampering & repudiation (`ledger`, `securities`, `posting`)

ID	Threat	STRIDE	L	I	Control(s)
T-21	A posted entry is edited or deleted through the app role, an ad-hoc SQL session, or `TRUNCATE`. `TRUNCATE` fires no row triggers and ignores RLS.	T / R	M	H	C-21, C-10 (no TRUNCATE grant), C-24 (a change outside the triggers breaks the chain)

ID	Threat	STRIDE	L	I	Control(s)
T-22	An unbalanced entry is committed.	T	L	H	C-22
T-23	Forged attribution: `actor_id` is set to another user.	R/S	M	M	C-23
T-24	The operator superuser disables triggers (`ALTER TABLE … DISABLE TRIGGER`, `session_replication_role = replica`) and edits history undetectably. The same applies to editing a restored backup.	T/R	L	H	C-24 (detects it, POLY-M2), R-1
T-80	The app role forges the journal sequence or the chain without a superuser. Paths: writing or pre-seeding an `entity_journal_counter` row; supplying its own `journal_number`; firing the deferred chain trigger early with `SET CONSTRAINTS … IMMEDIATE` so lines added afterwards are never hashed; or attaching a `poly_journal` trigger function to another table.	T/R	M	M	C-24 (`poly_journal` grants, counter created by trigger, line refused once chained), C-10, C-15
T-25	A manager's `auto_post` rule posts attacker-shaped entries without review.	T	L	M	C-17 (manager+), C-23 (`actor = rule:<id>`), C-53

3.4 TB-4 / TB-5 / TB-11 — Import, adapters & credentials

ID	Threat	STRIDE	L	I	Control(s)
T-30	A DB dump or backup leaks aggregator tokens, which are standing bearer access to real bank data.	I	M	H	C-30, C-72, C-73
T-31	Code execution or SSRF in the internet-facing `web` process opens every stored credential.	I / E	L	H	C-30 (`web` holds the public key only), C-31
T-32	Sealed-blob swap: someone with DB write copies the victim's sealed token into their own connection row, and the victim's bank data flows into the attacker's entity.	T / I	L	H	C-30 (AAD binds the blob to connection, entity and provider)
T-33	A token or URL leaks into logs, errors, API responses or the stored payload.	I	H	H	C-31, C-33, C-34, C-75
T-34	Unlink deletes the local row but leaves the token live at the provider.	I	M	M	C-32
T-35	A malicious CSV or OFX: XXE, billion-laughs, oversize input, spreadsheet-formula payloads.	T / D / I	M	M	C-40, C-41
T-36	Full account numbers stored in raw payloads widen the blast radius of any read leak.	I	H	M	C-34; payload readable by manager+ only (§4.2.1)
T-37	A malicious attachment (a PDF/HTML polyglot, a forged content type, a header-injecting file name) runs in our origin, or attachments fill the disk.	T / E / D	M	M	C-46

3.5 Worker & jobs (`apps/worker`, `packages/refdata`)

ID	Threat	STRIDE	L	I	Control(s)
T-40	A job with blanket cross-tenant write: one bug corrupts every tenant.	E / T	M	H	C-60 (every tenant write has a user principal)
T-41	A job keeps importing after the linking user loses their grant or deletes their account.	E / I	M	M	C-60 (`principal_invalid`), C-52
T-42	An unauthenticated caller triggers the Vercel Cron internal route.	E / D	M	M	C-61
T-43	Tampered or garbage reference data (prices, CPI) skews valuations.	T	L	M	TLS, source priority, C-45

3.6 TB-9 — Export, account deletion, purge & audit

ID	Threat	STRIDE	L	I	Control(s)
T-50	Purge abuse: an owner who is not the sole owner, `poly_app` setting the purge GUC directly, a stale export, or writes made after the export.	T / E	L	H	C-50
T-51	The purge path deletes nothing and reports success. This happens under FORCE RLS with no DELETE policy.	T	H if built per 8963efb	H	C-50 (`poly_purge` role-targeted DELETE policies; a zero-rows-remain assertion)
T-52	The purge erases its own evidence.	R	L	M	C-53
T-53	Export files linger on disk or in temp storage.	I	M	H	C-51
T-54	PRD §5's "rewrite the actor attribution" on account deletion would mean an UPDATE on append-only rows.	T	H	M	C-52 (tombstone the user row, never the entries)

ID	Threat	STRIDE	L	I	Control(s)
T-55	An exported CSV runs a spreadsheet formula planted in a payee string.	T / E	M	M	C-41
T-84	A password thief deletes an account that has no second step, purging its own Person's books, or is forced to enrol a factor first and so lands in T-82.	S / T	M	H	C-52 (password plus the emailed code, no factor set-up; §4.6)

3.7 TB-7 / TB-8 / TB-10 / TB-12 — Host, backups & supply chain

ID	Threat	STRIDE	L	I	Control(s)
T-60	Postgres or `web` is exposed publicly, because Docker-published ports bypass `ufw`.	I / E	M	H	C-70, C-71
T-61	A stolen disk, snapshot or backup exposes everything.	I	L	H	C-72, C-73
T-62	A box compromise also deletes or encrypts the off-box backups.	D / T	L	H	C-73 (append-only credential, immutable snapshots)
T-63	A compromised npm release (worm or typosquat) runs install scripts in CI or in the image.	E / I	M	H	C-80, C-82
T-64	A GitHub or Actions compromise pushes to prod through a deploy key.	E	L	H	C-81, C-83
T-65	Code merged but not yet released runs on staging on the prod kernel, then escapes its container.	E	L	H	C-70, C-74, R-4

ID	Threat	STRIDE	L	I	Control(s)
T-66	A secret is committed to the repo or baked into an image.	I	M	H	C-82, C-74
T-67	The local box, meant to be LAN-only, becomes reachable from the internet. Causes: a router UPnP auto-forward, a home router that allows inbound IPv6 to the box's global address, or a Docker-published port bypassing the host firewall.	I / E	M	H	C-76
T-68	The DNS-API token on the local box, used for the DNS-01 certificate, is stolen. The thief can issue certificates for the zone, or redirect the name for clients that resolve it publicly.	S / I	L	M	C-76 (token scoped to one zone, a file secret mounted into Caddy only); R-18
T-79	Private-CA mode on the local box. The operator's root certificate is installed on every device, so whoever holds its signing key can impersonate *any* website to those devices, banks included.	S / I	L	H	C-76: a name-constrained root, the root key offline, only an intermediate on the box. R-20.
T-69	The whole local box is stolen from the home.	I	L	H	C-72: the TPM unlock is sealed to Secure Boot (P8). The disk alone is unreadable elsewhere, and the whole box boots only to an authenticated login. C-73: backups are encrypted separately. R-21.
T-76	Staging, which now holds a keyring and sandbox tokens, is configured with prod's keyring, the production	I / E	L	H	C-77, C-74

ID	Threat	STRIDE	L	I	Control(s)
	Plaid secret or a real SimpleFIN token. Real bank access then lands on the box that runs unreleased code.				

3.8 TB-14 — Nesting, roll-up & cap table (`poly_rollup` , `poly_stake` , `poly_graph`)

ARCH §3.1a and §3.3a–c, on the Principal's rulings of 2026-11-06/07. Access stays per entity. These are the only paths by which structure or equity yields any figure about an entity the caller holds no membership on.

ID	Threat	STRIDE	L	I	Control(s)
T-70	A parent member with no membership on a child reads the child's rows (entries, lines, payees, memos, lots, positions) through a roll-up definer, the holdings popover, or a bug in a definer body.	I	M	H	C-25
T-71	Structure enumeration: a non-member reads `entity_rollup` rows, or ancestor names, beyond the ruled grant (members of the parent; the child's owner sees parent names only).	I	M	L	C-19, C-25
T-72	Time-slicing: calling the roll-up, economic or allocated-income definer for one-day periods, or comparing balances on	I	M	M	Consent disclosure (C-25): the inclusion step and the upward-consent notice state that any period is visible. No period guard, by the

ID	Threat	STRIDE	L	I	Control(s)
	consecutive days, rebuilds near-transaction-level detail of an entity the caller is not a member of.				Principal's ruling (P2, 2026-11-07). R-17.
T-73	The audience of an entity's totals grows without its owner knowing. It grows when the entity's parent is placed under another parent (consent travels upward), or when a member is added to an ancestor.	I	M	M	C-25: a placement notice and an ancestor-invite notice, both written by `poly_graph` triggers and both carrying the consent-disclosure sentence and the pull-out link. Also `app.parents_of` and pull-out. Notify, never block (P5, 2026-11-07).
T-74	The cap-table candidate list, or the holder notification, leaks names of Persons or entities to someone with no relation to them.	I	L	M	C-27
T-75	A holder's owners, who have no membership on the held entity, read other holders' rows or the held entity's books.	I	M	M	C-27
T-77	A cycle check or holder-graph check runs under a role that cannot see the whole graph, so a cycle through entities invisible to that role is accepted. Roll-ups then double-count, and allocations loop.	T	M	M	C-25 (graph triggers owned by `poly_graph`, with a graph-wide read)
T-78	A cycle or depth refusal reveals that a hidden link exists.	I	L	L	R-15. The inserter must own both ends, and the error names no entity.

4. Authentication & Authorization

§4.3–§4.5 are the rulings on ARCH OQ-2, OQ-3 and OQ-4. OQ-1 is ruled in §6.3. §12 has the summary and the changes against 8963efb.

4.1 Authentication

- **Provider.** BetterAuth in our own Postgres, via the Drizzle adapter. Sign-in methods:
  - Email + password, hashed with scrypt (the BetterAuth default).
  - OAuth 2.0 with PKCE and a verified `state`, via Google, Apple or GitHub.
  - Passkeys.
  - Optional magic links sent through Resend.
- **Invite-only registration (C-01).** The first user on an empty instance is the bootstrap user. After that:
  - Self-signup is off. New users arrive only through a membership invitation from an entity owner.
  - OAuth sign-in for an unknown email with no pending invite is rejected.
  - **No switch reopens public sign-up** (Principal, 2026-11-07, P6). No configuration flag or environment variable enables self-signup, so there is nothing to flip by mistake or through a configuration compromise.
  - **An invitation that shares no entity:** `polycarpic user invite <email>`.
    - Runs on the box only, by the operator, and writes an `audit_event`.
    - Creates a single-use invitation with the same expiry as an entity invitation.
    - The invitee gets only their own Person entity.
    - No entity is involved, so neither the co-owner approval (§4.2.2) nor the ancestor-invite notice applies.
- **MFA (C-03).** TOTP and passkeys, with single-use backup codes. BetterAuth stores the codes encrypted (XChaCha20-Poly1305 under the auth secret), not hashed. It checks a code by decrypting the list, so hashing them is not an option.
  - **Ruled (Principal, 2026-11-07, P1):** a second step (TOTP or a passkey) is required for any user holding owner or manager on any entity, with the one exemption below. It is optional for viewers (R-7). OAuth-only users must enrol a passkey or TOTP to hold those roles, because we cannot verify the IdP's MFA.
  - **The own Person is exempt until the first real action (Principal, 2026-11-08, OQ-34).** Every user owns their own Person from sign-up, so without an exemption the rule above would bind everyone, viewers included. The owner row on a user's own Person

(`entity.self_user_id`) is exempt while that Person is unshared: it has no other member, and it is not rolled up into a parent entity (A-28). Enrolment is forced, and the action is not performed, before any of these:

- creating a Trust, Business, Household or Holding (the creator's owner row is refused);
- sharing the own Person: the first invitation on it, or linking it under a parent, whose members would then see it (C-25);
- accepting an owner or manager invitation, or having a raise to manager or owner take effect (held, below);
- any step-up action on the own Person (export, purge, granting access, linking a bank), because step-up needs a second factor.

Viewing shared entities never triggers it. Unenrolled, the user can read and keep their own unshared books by hand, and nothing more. The refusals are in the database (BEFORE INSERT triggers on `membership` and `invitation`, ARCH §3.1c), not only in the UI.

- **The first factor is anchored on the account email (Principal, 2026-11-08; T-82).** A gate that asks for the password and then accepts the first factor offered stops only someone at an unlocked laptop. A password thief can do the same steps on their own device. So a user with no factor enrols their first one only after confirming a code sent to the account's email address, as well as the password. This applies at every OQ-34 gate, at the sign-in gate after an operator `reset-mfa`, and to an OAuth-only user, whose fresh IdP round trip stands in for the password.
  - **Order.** Password (or the fresh round trip), then the emailed code, then the factor. The code is asked for before any TOTP secret is shown or any passkey ceremony begins, so a thief learns nothing and stores nothing.
  - **The code.** Six digits from a CSPRNG, also offered as a link. It lasts 10 minutes and is used once. It is stored hashed as a T1 verification token and never logged. Five wrong entries void it. Sends are limited to 3 per 15 minutes per user (C-04). The response is the same whether or not a send was suppressed.
  - **Bound to the session.** The code, and the enrolment grant it opens, are keyed to the session that started the set-up. A code entered in another session, or after a new sign-in, is refused. The grant lasts 10 minutes and is consumed by the first factor write.
  - **Only the current, verified address.** Never a pending new address: changing email is itself a step-up action, so an unenrolled user enrols first, against the old address.
  - **Not completed until confirmed.** No TOTP row is marked verified, and no passkey row is written, without a live grant for that user and session. `second_step_at` stays null, and `app.promote_held_roles()` does not run. The gate is in the app, which wraps

BetterAuth's enrolment endpoints, and in the database (A-29), so a forgotten route fails closed.

- **Email is not a factor.** BetterAuth's `otp` two-factor method is not configured. An emailed code anchors the first enrolment, and never stands in for the second step.
- **Once a factor exists, nothing changes.** Adding a second method, and every step-up, uses the existing factor. The email anchor is for the first factor only.
- **Residual.** A thief with both the password and the email account can enrol. Email is already the recovery root (magic-link sign-in, invitations, the new-device notice in §9), so this adds no root. The user is emailed when a factor is enrolled (§9).
- **BetterAuth, measured (better-auth 1.7.18 and @better-auth/passkey 1.7.18, 2026-11-08, POLY-62 spike).** Where the docs and the code differ, the code was taken as correct. Paths are under `node_modules/better-auth/dist/`.
  - **No native email-confirmed enrolment.** The two-factor and passkey plugins have no such step. `emailOTP` offers only `email-verification`, `sign-in`, `forget-password` and `change-email`. Its codes are keyed on the email address, not the session, and `/email-otp/check-verification-otp` needs no session. It cannot be chained as a precondition, so the app's `email_challenge` and grant are the only path.
  - **Enabling two-factor** (`plugins/two-factor/index.mjs`). `/two-factor/enable` refuses a missing password (400) and a wrong one (`INVALID_PASSWORD`). It does not ask for a fresh session. With the `otp` method unconfigured, `method: "otp"` is refused (`OTP_NOT_CONFIGURED`). Enabling writes a `twoFactor` row with `verified = false`, an encrypted secret and encrypted backup codes, and leaves `user.twoFactorEnabled` false. It returns the TOTP URI and the backup codes at once, so the gate must sit on `/enable`. `skipVerificationOnEnable` defaults to false; the row is written with `verified = false`.
  - **Backup codes before TOTP is verified.** A signed-in session can redeem one at `/two-factor/verify-backup-code` (measured: 200), and nothing flips. BetterAuth's sign-in does not ask for a second step while `twoFactorEnabled` is false. So a backup code is never a step-up proof unless `second_step_at` is set (C-03).
  - **The first TOTP verify replaces the session.** `/two-factor/verify-totp` (`totp/index.mjs`) runs these steps in order: it sets `user.twoFactorEnabled`, creates a new session, deletes the requesting one, and then sets `twoFactor.verified`. When the last UPDATE runs, the requesting session no longer exists, and a grant that cascades from it is already deleted (measured). On BetterAuth's own pool these are four separate autocommits.

    - **Passkeys** (`@better-auth/passkey`). There is no `/passkey/add-passkey` . Registration is `/passkey/generate-register-options` and `/passkey/verify-registration` . Both require a session no older than `session.freshAge` (default 24 hours; a 25-hour-old session got `SESSION_NOT_FRESH`), and no password. "Fresh" means the session's age, not a re-authentication. A session-only call wrote a passkey row (measured, with a software authenticator). `registration.afterVerification` runs before the row is written and can refuse.
    - **Hooks.** `databaseHooks` run only for the core models: `user` , `session` , `account` and `verification` . The plugins write `twoFactor` and `passkey` through the raw adapter, and no hook ran for either (measured). So a database hook cannot be the enrolment gate. A global `hooks.before` sees the path and the session id, and when it throws, nothing is written (measured on `/two-factor/enable` and on passkey options). It runs in the process, not in a database transaction.
    - **Connection.** On its own pool, BetterAuth's `twoFactor` INSERT ran on a different backend and transaction from our open `withTenant` , and the trigger read `app.session_id` as null (measured). BetterAuth accepts any object with `connect()` as a Postgres pool. A 30-line routing pool hands it our open transaction's client, and turns its `BEGIN` / `COMMIT` into savepoints. With that pool, the `twoFactor` and `passkey` triggers saw the GUC on our backend and in our transaction. Rolling ours back removed BetterAuth's row. A trigger that refuses without the GUC blocked a write outside `withTenant` and allowed one inside it. The first-verify sequence ran inside one transaction, and passkey registration with `createSession` ran through savepoints (all measured). This is A-31.
    - **Passwordless users.** By default `/two-factor/enable` always asks for a password (`utils/password.mjs` `shouldRequirePassword` ; code-read). An OAuth-only user could not enrol TOTP. `allowPasswordless: true` skips the password only for users with no credential password. The before-hook must then require the fresh IdP round trip (A-33).
- **A raised role waits for enrolment (v1.2; mechanism ARCH §3.1c).** When a member without a second step is raised to manager or owner, `membership.role` stays the role in force (viewer), and the raise is parked in `membership.pending_role` , with who raised it and, on the approval path, the applying `change_request` . Nothing that authorizes reads `pending_role` : `has_role` , the RLS policies, `app.owner_count` , the §4.2.2 approver set, the 1 → 2 transition and founder eligibility all see viewer, so a check that forgets the pending state fails closed (C-03, T-81). A-25 is met:
    - `user.second_step_at` is written only by a `poly_govern` definer trigger that recomputes it from BetterAuth's factor tables. `poly_app` cannot set it.

        - The only write that makes the role effective is `app.promote_held_roles()` (`poly_govern`), in the enrolment's own transaction. It re-checks for a factor row, changes only the role columns, and writes a `role.promoted` `audit_event`. An approved co-owner raise is not re-approved.
        - An INSERT of manager or owner for an unenrolled user is refused, so an owner or manager invitee enrols before the row exists and is never a held viewer.
        - Any owner may cancel a held raise as a lowering, with no approval. A held raise does not expire.

      The edges:

        - **A new owner never inherits a raise they had no vote on.** At the 1 → 2 transition, a held raise the sole owner made directly is cancelled along with the direct invitations, in the same transaction and under the same entity-row lock (C-28), and its raiser is notified. A held raise applied through `app.apply_change` stays, as an invitation applied that way does, because the owners at the time approved it.
        - **The last method cannot be removed** while the user holds manager or owner outside the own-Person exemption. The operator's audited `reset-mfa` is the only way past. It revokes every session, and the sign-in gate then reaches no route but Set up a second step until they enrol.
        - A held owner is not eligible for a founder transfer, and the account-deletion transfer offers only enrolled members (§4.6).
    - 8963efb recommended MFA-optional, fearing lockout with no support desk. The lockout risk has a cheaper answer: an operator-side CLI (`polycarpic user reset-mfa <email>`). It runs only on the box and writes an `audit_event`. The operator is already the trust root (R-1).
- **Step-up.** These actions require a re-authentication within the last 5 minutes (password + MFA, or a passkey):
    - purge
    - entity export
    - granting a membership, changing a role, or cancelling a held raise
    - requesting, approving, declining or forcing a co-owner request (§4.2.2)
    - editing the cap table
    - transferring the founding-owner role, or stepping down from it
    - linking, relinking or unlinking a provider
    - removing a second-step method, or disabling MFA
    - changing email

    - deleting the account

  Step-up always takes the second factor. A user with none who starts one of these actions, on their own Person included, is sent to Set up a second step first, and the action is not performed (OQ-34). The one exception is deleting an account that has no factor (§4.6).

    - **The window is per session.** `requireRecentAuth()` reads a `recent_auth_at` stamp on the server-side session record, set by a successful step-up. It does not read the session's creation time.
    - **Ruled (v1.2.2): a completed first enrolment counts as a step-up.** It has checked the password (or a fresh IdP round trip), the emailed code and the new factor's own proof, in that session. That is more than Confirm it's you asks. So it sets `recent_auth_at`, and the gated action that sent the user to set-up continues without asking again, within the same 5 minutes. A failed or abandoned set-up sets nothing.
    - The unenrolled deletion stamp (§4.6) is narrower. It authorises only that deletion run, and no other step-up action.
- **Web sessions (C-02).** Server-side BetterAuth records.
    - The cookie is `__Host-`-prefixed, `HttpOnly`, `Secure` and `SameSite=Lax`.
    - Sessions expire after 7 days idle or 30 days absolute.
    - All sessions are revoked on password change, MFA reset or account deletion.
    - Membership changes need no session revocation, because visibility is recomputed from `membership` in every transaction.
- **Native and MCP tokens (later).** Named, listable, individually revocable bearer tokens, stored hashed in our own table. BetterAuth's `bearer` plugin is not this: its token is the session token, stored in the clear (R-12). Without `requireSignature: true` it accepts the unsigned token too (`plugins/bearer/index.mjs`; code-read). If the plugin is ever enabled, it is enabled with `requireSignature: true`.
    - Stored hashed as Tier 1, and kept in the device Keychain.
    - Scoped to a *user*, never to an entity. A token grants what its user can do at the time of each request.
- **No in-app super-admin.** No application role reads across tenants. The operator's superuser is the only such principal, and it sits outside the app (R-1).
- **Service authentication.**
    - Each service uses its own Postgres role and password, delivered as files (C-74): `web` uses `poly_app`, `worker` uses `poly_worker`, migrate uses `poly_migrator`.
    - `web` and `worker` never talk over HTTP; they coordinate through pg-boss tables.

- On Vercel, Cron calls an internal route authenticated by `Authorization: Bearer <CRON_SECRET>`, compared with `timingSafeEqual`. The route is compiled out of the `adapter-node` build (C-61).

4.2 Authorization model

RBAC over the membership join (`user`, `entity`, `role`), enforced by Postgres RLS. A user's rights come only from `membership` rows. They never come from a `user_id` column on a data table, and never from entity structure. Access is a grant; the hierarchy grants nothing.

`can(role, action)` in `packages/tenancy` exists to render a correct UI and to return a clean 403 early. It is *not* an enforcement point.

The design test: delete every `can()` call, and the system must still be safe. Any control that fails this test is app-layer-only authorization, and the security gate rejects it.

There are two exceptions, and both are enforced in database functions rather than `can()`: step-up recency (checked by the route, and required as an argument by the purge function) and sole-owner status.

4.2.1 Permission matrix (owner / manager / viewer)

Roles are the same for every entity type. The type drives tax routing and reporting, never authorization. The matrix is stored as data in `packages/tenancy`. Every row gets a pgTAP assertion that bypasses the app layer, plus an endpoint-level test (C-17).

Action	viewer	manager	owner	Enforced by
Read dashboard, balances, posted entries, lots, reports, drafts	✓	✓	✓	SELECT policy: `entity_id = any ((select app.visible_entities()))`
Read linked-	✓	✓	✓	Full numbers are never stored (C-34), so there is nothing to mask per role.

Action	viewer	manager	owner	Enforced by
account name and last-4				
Read the raw provider payload (`raw_transaction.payload`)	×	✓	✓	Policy or column grant through a manager+ view. The payload is the least-curated, most PII-dense field we hold.
Read the entity's membership list	own row	✓	✓	SELECT policy on `membership`
Create, edit or delete a draft; confirm a draft (post); reverse an entry	×	✓	✓	Per-command policies with `WITH CHECK app.has_role(entity_id,'manager')`
Edit the chart of accounts; create or edit rules, including `auto_post`	×	✓	✓	As above. `auto_post` creation writes an `audit_event`.
Upload CSV / OFX	×	✓	✓	Insert policy on `import_batch` and `raw_transaction`

Action	viewer	manager	owner	Enforced by
Link a provider connection	×	✓ step-up	✓ step-up	Insert policy on `provider_connection`. The linking user becomes `linked_by_user_id`.
Unlink a connection	×	own links only	any, step-up	Policy compares `linked_by_user_id`. Revocation runs as a worker job (C-32).
Sidebar-IA actions (ARCH §2.4 @ 884748c). G6 in ARCH §3.2a relies on these rows.				
Refresh a provider token (relink)	×	✓ step-up, own links	✓ step-up	A relink stores a new sealed credential, so it is treated like a link (C-30). The policy compares `linked_by_user_id` for managers. Writes an `audit_event`.
Map a linked account to a custodial GL account	×	✓	✓	UPDATE policy on `linked_account`: manager+. The target account must belong to the same entity (composite FK). A mapping change writes an `audit_event`, because it redirects where bank data posts.
Sync now	×	✓	✓	Enqueues a worker job and never opens the credential in `web` (TB-4). Rate-limited per connection (minimum interval set in INFRA) against provider quota and cost.

Action	viewer	manager	owner	Enforced by
Post, Split, Group, + New entry, Correct entry	×	✓	✓	G6: `WITH CHECK app.has_role(entity_id,'manager')` on `journal_entry` / `journal_line`, with the actor forced by trigger (C-23). Correct entry is a reversal plus a new entry and never an edit (C-21). G1, G2, G4 and G5 are re-checked by the database at commit.
Rule auto-post opt-in (opting in counts as confirmation)	×	✓	✓	The opting-in user is recorded on the rule (`auto_post_by_user_id`) and is the principal for every auto-post, as in OQ-3. If that user drops below manager or is deleted, `auto_post` flips off and an `audit_event` is written, mirroring `principal_invalid`. Auto-posts pass G1–G6 like any post. Opt-in writes an `audit_event` and emails the owners.
Per-entity import schedule	×	✓	✓	UPDATE policy on the entity's import settings: manager+. The interval is bounded below by the same INFRA minimum as Sync now.
Set entity metadata (name, tax classification)	×	×	✓	UPDATE policy on `entity`: owner
Set a parent link	×	×	✓ on child *and* parent	C-19. The `poly_graph` trigger refuses cycles and links past depth 8, and notifies every owner beneath (C-25).
Remove a parent link	×	×	✓ on child *or* parent	DELETE policy on `entity_rollup` (Principal, 2026-11-06). Writes an `audit_event`.

Action	viewer	manager	owner	Enforced by
Read parent links; see the parents above my entity	links where a member of the parent	as viewer	as viewer, plus `app.parents_of` on an owned child	SELECT policy on `entity_rollup`: any membership on the parent, or owner on the child. `app.parents_of` returns ancestor ids and names only (C-19).
Read a parent's consolidated totals (POLY-1.1)	✓	✓	✓	`app.rollup_totals` / `app.rollup_economic`, guarded by viewer+ on the parent. Aggregates only, never rows (C-25).
Read the cap table	✓	✓	✓	SELECT policy on `equity_total` / `equity_holding` by `entity_id`. A holder's owners who are not members use `app.my_stakes()` (C-27).
Edit the cap table; read holder candidates	×	×	✓ step-up; co-owner approval if more than one owner	INSERT policy on `equity_*`: sole owner only. With more than one owner, the edit is held and applied only through `app.apply_change` (§4.2.2, C-28). `app.cap_table_candidates` is owner-only (C-27).
Invite a member, raise a role, add an owner	×	×	✓ step-up; co-owner approval if more than one owner	INSERT and UPDATE policies on `membership` (and `invitation`): sole owner only. Otherwise the change goes through §4.2.2. The invite email is sent only once the change applies (C-20, C-28).
Remove or demote a co-owner	×	×	✓ step-up; every other owner approves, the target included. After a	Through §4.2.2 only (`app.apply_change`, or `app.force_change` for the founder). A last-owner trigger also applies (C-20). The founder cannot be removed or demoted this way.

Action	viewer	manager	owner	Enforced by
			decline or expiry, the founding owner may force it.	
Transfer the founding-owner role, or step down from it	×	×	founder only, step-up	An UPDATE of `entity.founding_owner_user_id` through a `poly_govern` definer. The new founder must be a current owner. Every owner is notified, and an `audit_event` is written (§4.2.2).
Remove a viewer or manager; lower or give up one's own role	×	×	✓ step-up (own role: self, any role)	A single-owner action (Principal, P5b). DELETE and UPDATE policies on `membership`, plus the last-owner trigger (C-20).
Export the entity's complete books	×	×	✓ step-up	PRD §5 grants this to the owner. It is exfiltration-shaped, so it is audited (C-51). *Revised:* 8963efb allowed manager+.
Purge the entity's books	×	×	sole owner, step-up	C-50 (§4.5)
Read the entity's `audit_event` rows	×	×	✓	SELECT policy: owner, or `actor_id = app.current_user_id()`
Clone or edit own report definitions;	self	self	self	Owner-user policies. Account deletion resolves every solely-owned entity in the flow, by transfer or purge, or the user aborts. The

Action	viewer	manager	owner	Enforced by
export own personal data; delete own account				user is the only invoker (§4.6, C-52). *Revised in v1.1:* deletion was refused for a sole owner.

4.2.2 Co-owner approval on multi-owner entities (Principal, 2026-11-07, P5b)

Co-trustees and partners govern as a group. Without this rule, any one owner, including a 0-unit bookkeeper owner, could admit anyone, raise a role or rewrite ownership alone (T-26). On an entity with more than one owner, these changes need the approval of **every other current owner**:

- (a) inviting a member, or raising a role (viewer → manager → owner);
- (b) removing or demoting a co-owner;
- (c) any cap-table edit (`equity_total`, `equity_holding`).

Removing a viewer or manager, and lowering or giving up one's own role, stay single-owner actions. Sole-owner entities are unaffected.

- **A pending state, enforced in the database.**
  - **The request.** A `change_request` row holds entity, kind, payload, requester, created time, expiry and status (`pending | applied | declined | expired | withdrawn`). One `change_approval` row is written per approving owner, and both tables are append-only apart from the status.
  - **Direct writes.** `poly_app`'s direct INSERT and UPDATE on `membership`, `invitation` and `equity_*`, for the actions above, pass only while `app.owner_count(entity_id) = 1`.
  - **Multi-owner path.** Otherwise the only write path is `app.apply_change(change_id)`. It is an allowlisted definer owned by a new NOLOGIN, NOBYPASSRLS `poly_govern` role, holding role-targeted INSERT and UPDATE on exactly those tables. A `can()`-only gate fails the §4.2 design test.
- **Who approves.** The approvers are every current owner other than the requester, recomputed when the change is applied, never fixed when it is requested.
  - For a co-owner's removal or demotion, the target is one of the approvers, so no owner can be removed by the others alone. A deadlock is broken by the founding owner (below).

    - An approval counts only if its approver is still an owner at apply time.
    - One decline ends the request.
    - The requester may withdraw it.
- **Applying.** When the last approval lands, `apply_change` runs in that approver's transaction and re-validates the payload against current state:
    - the role still exists;
    - the cap-table Σ invariant holds;
    - the entity version the request was made against is unchanged, or there is no conflicting applied change since.

  If validation fails, the request is declined as stale and nothing applies. Only on apply does the invitation become live and its email go out.
- **Step-up.** Requesting, approving and declining each need a re-authentication within 5 minutes, with the second factor required for owners (C-03, P1).
- **Audit and notice.** Each step writes an `audit_event`: `change.requested`, `.approved`, `.declined`, `.withdrawn`, `.expired` and `.applied`. Every owner is notified on request and on outcome. Pending requests, including an invitee's email address, are visible to owners only.
- **The founding owner (Principal, 2026-11-07, R-19 ruling).** Each entity records one founding owner, `entity.founding_owner_user_id`, set to its creator. For a Person entity that is its own user.
    - **Always an owner.** A trigger refuses any state where the founder does not hold owner.
    - **Breaking a removal deadlock.** Once a request to remove or demote a co-owner is `declined` or `expired`, the founder may force it through `app.force_change(change_id)`, a `poly_govern` definer.
        - It requires that the caller is the founder, that the kind is a co-owner removal or demotion, and that the status is declined or expired.
        - It requires a step-up within 5 minutes, with the second factor.
        - It writes a `change.forced` `audit_event` and notifies every owner, the target included.
        - The founder may raise the request themselves. The force still needs the decline or the 14-day expiry first, so the target always has warning.
    - **No other extra power.** The founder has none over invites, role raises or cap-table edits, which still need every other owner. The founder is never a target of `force_change`.
    - **Leaving the role.** The founder can only step down or transfer it in-app, as a founder-only action, with step-up and a notice to every owner.
        - Stepping down names a successor among the current owners.

        - A founder who would give up their own owner role must transfer the founder role first, if another owner exists.
    - **Account deletion.**
        - In the in-flow deletion (§4.6), a founder of an entity with other owners must name a successor among them. The tombstone transaction refuses while any entity still names them as founder.
        - On a sole-owned entity, the transfer target becomes founder. On a purge, the question does not arise.
    - **A founder who vanishes without deleting.** The backstop is the operator CLI (`polycarpic entity set-founder`, `polycarpic membership …`), which is audited and notifies every owner.
- **Edges** (SecEng rulings):
    - **An owner who never responds.**
        - A request expires 14 days after it is made, and expired counts as declined. Silence never approves.
        - Owners are reminded at day 7.
        - The requester may ask again. One open request per kind and target, and an expired request can be re-raised at most once per 7 days.
    - **An approver in the middle of account deletion.**
        - While their deletion flow runs they are still an owner and may approve or decline.
        - At their tombstone they stop being an owner, so their approvals stop counting and they stop being required.
        - Each pending request on their entities is re-evaluated. If it is now fully approved, it is *not* auto-applied: the requester is notified to confirm, with step-up.
        - Leaving an entity is giving up one's own role, so it needs no approval.
    - **The owner set changes while a request is pending.**
        - A new owner becomes a required approver.
        - A departed owner drops out, and the requester-confirm rule above applies.
    - **Bootstrap, when the second owner is added.**
        - A sole owner adds the first co-owner, by invite or by raising a member's role, as a single-owner action. There is nobody to approve.
        - The invitee's acceptance is their own consent, and they do not approve their own admission.
        - From the moment two owners exist, every further change in (a) to (c) needs approval.

        - **Invitations sent while sole owner (OQ-31, confirmed).** When the effective owner count goes from 1 to 2, every outstanding invitation the sole owner issued on the direct path is revoked, at every role, and its inviter is notified to re-raise it as a request. Otherwise an owner invitation issued alone could land after the approval rule applies.
            - An invitation records how it was issued (direct, or the `change_request` that applied it), and that column is not updatable. Approved invitations are not revoked.
            - The revocation runs in the same transaction as the membership write that makes the second owner, not in a later job. That write locks the entity row before counting owners, so two direct owner invitations accepted at once serialize: the first makes two owners and revokes the second.
            - A revoked token is refused at acceptance with the same response as an expired one.
            - The rule fires on every 1 → 2 transition, including after an entity drops back to one owner. A raised owner still waiting for enrolment (§4.1) does not count until the role is effective. The transition, and the revocation, happen at enrolment.
        - The entity-creation default cap table, and transfer in account deletion, are sole-owner cases.
    - **Pending changes and the purge.** A purge needs a sole owner, so §4.2.2 never meets it. Pending requests on a purged entity are deleted with it, and their audit events survive.

pgTAP for the sidebar-IA rows (qa-engineer, C-17). Each row gets an allow test and a deny test, run as `poly_app` with the app layer bypassed:

- A viewer cannot run any of the actions: every INSERT or UPDATE is rejected by policy.
- A manager cannot relink or unlink another user's connection.
- A mapping to another entity's account fails the composite FK.
- An auto-post whose opting-in user has been demoted to viewer writes 0 rows, and its rule ends with `auto_post = false`.
- A "Correct entry" never UPDATEs the original; it leaves one reversal plus one new entry.
- Sync and schedule below the minimum interval are rejected.

Revocation semantics. Revoking a membership takes effect on the next transaction; there is no cached authorization. The user's sessions stay valid for their other entities. Data a client has already fetched cannot be recalled (R-11). The native milestone must purge its cache on any 401, 403 or empty scope.

4.3 RLS mechanism — ruling on ARCH §3.3 (OQ-2)

Ruling: confirmed, with amendments. These stand as designed: transaction-local GUC, STABLE SECURITY DEFINER visibility functions, ENABLE + FORCE on every tenant table, and default-deny.

Additions (a)–(d) are already in ARCH (from 8963efb) and remain in force:

- (a) The coverage probe is derived from the catalog.
- (b) No CREATE on `public`.
- (c) Per-command policies with `WITH CHECK` on every write; `FOR ALL` is banned.
- (d) The inversion canary stays.

Review found defects that (a)–(d) do not cover. Amendments (e)–(l) are new:

e. **FORCE RLS vs. the definer functions (latent defect, T-20).**
   - *The problem.* Under `FORCE`, the table *owner* is also subject to policies. ARCH has `visible_entities()` and `has_role()` owned by `poly_owner`, and they read `membership`, which is itself FORCEd.
   - *Why it breaks.* Once `membership`'s own SELECT policy calls `has_role()` (so that managers can list members), the chain function → `membership` policy → function makes Postgres raise "infinite recursion detected in policy". If the policy does not call it, the functions silently see only the rows the policy allows.
   - *Fix.* A `poly_authz` role (NOLOGIN, NOBYPASSRLS) with SELECT on `membership` and `entity_type` only, plus a role-targeted policy `CREATE POLICY authz_read ON membership FOR SELECT TO poly_authz USING (true)`. Both functions are owned by `poly_authz`. FORCE stays uniform and nothing gets BYPASSRLS. `poly_app` cannot use the policy, because it is not a member of `poly_authz`.

f. **The empty-string GUC (defect).**
   - *The problem.* 8963efb and ARCH say `current_setting('app.user_id', true)` returns NULL when unset. That is only true until the GUC is first touched in a session. After any transaction-local set, the placeholder reads back as `''`, and `''::uuid` raises. That is fail-closed, but noisy and fragile.
   - *Fix.* Every policy reads `app.current_user_id()`, which is `nullif(current_setting('app.user_id', true), '')::uuid`. The pool-reuse test runs its no-context probe on a connection that has already carried a context.

g. **`TRUNCATE` (defect).** It ignores RLS and fires no row-level DELETE triggers. *Fix:* a statement-level `BEFORE TRUNCATE` trigger that raises on every append-only table, and no login role holds TRUNCATE.

h. **Pinned search_path.**

    - Definer functions use `SET search_path = ''`, and every name is schema-qualified.
    - EXECUTE is revoked from PUBLIC and granted explicitly.
    - Policies call these functions as `(select app.fn())`, so each is evaluated once per statement as an initplan.
    - A catalog allowlist names every SECURITY DEFINER function, and any function not on it fails the battery.

i. **Immutable tenancy columns.** `entity_id`, `actor_kind` and `actor_id` are left out of `poly_app`'s column-level UPDATE grants, so no row can change tenant or author. UPDATE policies carry both `USING` and `WITH CHECK`.

j. **Views.** Every view over tenant tables is created `WITH (security_invoker = true)`. Materialized views over tenant data are banned, because RLS cannot apply to them. The battery asserts both rules.

k. **Role hygiene.** `poly_app` and `poly_worker` are NOSUPERUSER, NOBYPASSRLS and NOCREATEROLE. They have no TRUNCATE, REFERENCES or TRIGGER grants and are members of no other role. The battery checks this with `pg_has_role` and `has_table_privilege`.

l. **Non-enumerable identifiers.** Primary keys for tenant-visible rows are UUIDs (v7 is acceptable). FK checks bypass RLS, so sequential IDs would make an existence oracle. ARCH §4.1's `/entities/42` becomes a UUID.

Pool safety. `set_config(…, true)` is safe under the in-process node-postgres pool and under transaction-mode pgbouncer, including Neon's pooler. The rules that keep it safe:

- Statement-mode pooling is banned, because it breaks transactions.
- The deploy checklist asserts the pooler mode on both targets (DevOps).
- `withTenant()` refuses to nest (AsyncLocalStorage guard).
- `withTenant()` accepts only a branded `SessionUserId`, which only `packages/auth` can construct.
- The raw pool is not exported, and the layer lint enforces that.
- Session-level `SET` is banned by lint.

What RLS buys, and what it does not.

- *It buys:* authorization bugs become empty results instead of leaks. That covers a missing `WHERE`, an IDOR, a forgotten table.
- *It does not buy:* any defence against SQL injection or code execution inside `poly_app`. The GUC is app-asserted, so anyone who can run SQL as `poly_app` can set it to anyone (R-2).

- *For that case:* prevention is C-44. The blast radius is capped by C-30 (`web` cannot open credentials) and C-10 (no DDL, no TRUNCATE).

4.4 System jobs — ruling on OQ-3

Ruling: confirmed as folded into ARCH. A job's tenant writes run under the linking user's membership. No tenant policy has a system bypass clause. The requirements already on POLY-64 stand:

- `provider_connection.linked_by_user_id` is NOT NULL.
- `withSystem(job, connection, fn)` sets `app.user_id` to that user and `app.actor = system:<job>`. Authority and attribution stay separate.
- If that user's membership drops below manager, or their account is deleted, the connection flips to `principal_invalid`. That state surfaces on the dashboard, writes an `audit_event` and notifies the owners. It never fails silently.
- A job never falls back to running context-free. If no principal resolves, it errors.

Additions:

- **Entity-bound jobs with no connection** (the tax-period freeze) run under a current owner of the entity, chosen deterministically as the earliest owner membership, with `actor = system:<job>`. Every tenant write therefore has a human principal.
- **Scheduling** needs a cross-tenant list of targets. That list comes from one allowlisted SECURITY DEFINER function, `app.job_targets(job)`, which returns only (`entity_id`, `principal_user_id`, `connection_id`). Only `poly_worker` can execute it.
- **Reference-data writes** (prices, CPI, tax tables) go through role-targeted policies (`TO poly_worker`). `poly_app` gets SELECT only. A role-targeted policy cannot be unlocked by any GUC that `poly_app` sets.
- **Cash-in-Transit aging** is computed on read by the exceptions report, not stored by a job. That removes a job that would need cross-tenant reads.
- **`poly_worker` tenant DML** is limited to the import path:
  - `import_batch`, `raw_transaction`, `reported_balance`
  - `draft`, `transfer_match`
  - status columns of `provider_connection`
  - `journal_entry` / `journal_line`, for `auto_post` only
  - `tax_period_freeze`, `audit_event`

4.5 The purge path — ruling on OQ-4 (revised)

Ruling: the purge function is owned by a dedicated NOLOGIN `poly_purge` role, with role-targeted DELETE policies. This reverses 8963efb, which had the function owned by `poly_owner` and rejected a separate role. The guard order ARCH folded in is kept, with two guards added.

- **Why 8963efb's shape fails (T-51).**
  - A definer function owned by `poly_owner` runs as the table owner, and under `FORCE ROW LEVEL SECURITY` the owner is subject to policies.
  - Append-only tables deliberately have no DELETE policy. So the function would delete *zero rows and report success*.
  - Adding a `poly_owner` DELETE policy would instead hand the purge capability to the migration owner.
- **Why the separate-role objection does not hold.** 8963efb said "a role needs a credential". A NOLOGIN role has no credential; it is only the owner identity for the definer function. The entry point is just as narrow: one function, with EXECUTE granted to `poly_app`. The only change is which role's privileges the function runs with.
- **Shape.**
  - `poly_purge` is NOLOGIN and NOBYPASSRLS, with DELETE on the purgeable tables only.
  - Each purgeable table gets `CREATE POLICY purge ON <t> FOR DELETE TO poly_purge USING (entity_id = nullif(current_setting('app.purge_entity', true), '')::uuid)`.
  - The append-only triggers allow DELETE only when `current_user = 'poly_purge'` *and* `OLD.entity_id` matches the GUC. `poly_app` can set the GUC but can never become `poly_purge`.
  - `app.purge_entity(entity_id, export_id, reauth_at)` is SECURITY DEFINER, owned by `poly_purge`, with `search_path = ''`.
- **Guards, in order, in one transaction:**
  1. Re-authentication within 5 minutes. The route checks it, and the function requires the timestamp as an argument.
  2. Sole-owner check against `app.current_user_id()`.
  3. An `export.completed` audit event for this entity, by this user, within 24 hours, carrying a SHA-256.
  4. *(New.)* No tenant write to the entity since the export's snapshot marker, so a manager's later postings cannot slip past the export. The entity row is locked `FOR UPDATE`.

5. An `entity.purged` `audit_event` is written before the first delete. It records the actor, entity id and name, the export event, per-table row counts and timestamps. `audit_event` has no FK to `entity` and no purge policy.
6. Every connection is revoked at the provider. The worker revoke job must have succeeded; a failed revocation blocks the purge and is never skipped.
7. Set `app.purge_entity` transaction-locally and delete in dependency order.
8. *(New.)* Assert zero rows remain for the entity in every purgeable table before commit. This is the fail-loud guard against T-51.

- **Not permitted:**
  - purge by a job;
  - purge through the API without interactive, typed confirmation of the entity name;
  - purge of a partial date range (that is an edit, and edits are reversals);
  - purge while a second owner exists.
- **Backups:** purged data persists until the backup retention window rolls (R-5). The purge warning says so.
- **Rejected alternatives:**
  - `ALTER TABLE … DISABLE TRIGGER` is DDL. It needs an ACCESS EXCLUSIVE lock and the table owner, and it affects every tenant.
  - Tombstoning is not a purge, so it breaks the PRD §5 promise.
  - BYPASSRLS is broader than needed.
- **No execution delay (Principal, 2026-11-06, OQ-23).** SecEng had recommended a cancellable 24-hour delay against a hijacked session. The Principal ruled it out, after hearing that rationale and the architect's objections.
  - The purge runs as soon as the guards pass.
  - Every member is emailed immediately, before the entity's rows go.
  - Recorded as R-16.
- **Roll-up junction.** The `entity_rollup` purge policy matches either column, so purging an entity removes it from every parent and the parents' owners are notified (ARCH §3.4). The cap-table rows on *other* entities that name the purged entity as a holder are open: see A-19.

4.6 Account deletion in-flow — who invokes the purge (OQ-25)

Ruling: the account being deleted is the only invoker of every step. This follows ARCH §3.4a, on the Principal's ruling of 2026-11-06, which replaced the refusal for a sole owner.

- **Who.** Only the user who is deleting their account, signed in, with a fresh step-up:

    - password plus their second factor, or a passkey;
    - an OAuth-only user does a forced fresh round trip (`prompt=login` , `max_age=0`), plus their second factor (required for an owner, P1);
    - **a user with no second step** (Principal, 2026-11-08): the password, or the fresh round trip, plus a code sent to the account's email, with the same lifetime, attempt cap, send limit and session binding as the first-enrolment code (§4.1). No factor is set up. A departing user gains nothing from a factor they would use once, and a password thief without the mailbox can neither delete nor enrol (T-84, T-82). The confirmation authorises this deletion run only, including the own Person's export it produces, and no other step-up action.

  An enrolled user's deletion is unchanged. An unenrolled user can hold no manager or owner role outside their unshared own Person, so their run is that Person's purge (with its export), their viewer memberships ended, and any held raise cancelled. The plan, transfer-or-purge choices, exports and typed name are otherwise unchanged. Never a job, never another member, never the operator CLI. The user still holds owner when each purge runs, so the §4.5 sole-owner guard passes on its own terms, and no guard is relaxed for this flow.
- **The plan is bound to the session.** The per-entity choices (transfer to whom, purge with which export) are held server-side, keyed to the session that made them.
    - Another session, or a new sign-in, starts again from the list.
    - If the 5-minute step-up lapses mid-run, the user is prompted again. The run never continues silently.
- **Founder succession.** For each entity where the user is the founding owner and other owners remain, the flow requires a successor named among them (§4.2.2). A sole-owned entity's transfer target becomes founder.
- **Exports.** Every purge choice carries its own `export.completed` event, by this user, within 24 hours (guard 3). The flow produces these exports before the final confirmation.
- **Order.** Transfers first, then purges, then the tombstone.
    - A transfer names a current member who is not mid-deletion themselves. The recipient is notified, and the transfer writes an `audit_event` .
    - A failed provider revocation stops the run at that entity (§4.5 guard 6). Nothing later is applied.
- **Audit.** One `account.deleted` `audit_event` records every choice: the transfer targets, and each purged entity's id with its export SHA-256. It sits alongside the per-entity `entity.purged` events. All of them keep the tombstoned user id.
- **Email.** Every member of a purged entity is emailed as each purge begins (OQ-23). The departing user gets a confirmation at their address before the tombstone removes it. Every remaining member of a shared entity they leave is notified (ARCH §3.4a).

- **Tests** (C-52):
  - another member cannot drive a deletion's purge step;
  - a plan made in session A cannot be confirmed from session B;
  - a lapsed step-up blocks the next purge;
  - an unenrolled user's deletion needs the password and the emailed code, offers no factor set-up, and is refused with the password alone;
  - the unenrolled deletion confirmation does not pass step-up for any other action;
  - a failed revoke leaves the account undeleted and the remaining entities untouched;
  - the tombstone transaction refuses if a sole-owned entity appeared during the flow.

5. Data Classification & Handling

User data falls into three sensitive tiers (from the Research brief), plus two non-tenant classes. Tiers 2 and 3 together are "sensitive personal financial data". Memo and payee text routinely leaks medical, income and relationship details, so it is not "just transactions".

Everything sits on an encrypted volume (C-72) and travels only over TLS. The table covers handling beyond that baseline. Assigning a tier to a new field is a security-gate question.

Tier	Data	At rest (beyond volume encryption)	Exposure rules	Retention
T1 Authentication secrets — Restricted	Password hashes, TOTP seeds, backup codes, session records, native/MCP bearer tokens, verification and reset tokens, IdP OAuth tokens	Passwords use scrypt. Our own bearer tokens and verification codes are hashed. Backup codes and TOTP seeds are encrypted with XChaCha20-Poly1305 under the auth secret. Session tokens are stored in the clear (measured, 1.7.18; R-12). `encryptOAuthTokens: true` encrypts the access and refresh tokens; it is off by default (measured). It does not cover the ID token, which a `databaseHooks.account` before-hook drops (measured on create; update is set the same way). We request no offline OAuth scopes, because we need identity only.	Never logged. Never returned after creation. Never in a URL, except single-use reset and magic links that expire after 15 minutes.	Sessions: 7 days idle, 30 days absolute. Tokens: until used or expired. All deleted on account deletion.
T2 Standing bank-access credentials — Restricted	SimpleFIN access URLs, Plaid access tokens, broker refresh tokens	HPKE-sealed, with AAD binding (§6.3, C-30). `poly_app` has no column SELECT (C-31).	Never sent to any client in any form, not even masked or partial. Never in DTOs, logs or the raw payload. Opened only in `worker`.	Until unlink. Revoked at the provider on unlink, account deletion or purge (C-32).

Tier	Data	At rest (beyond volume encryption)	Exposure rules	Retention
(highest value)				
T3 Financial data and PII — Confidential	Balances, transactions, payees, memos, holdings, lots, entries, tax figures, entity names and classification, account last-4, raw payloads (redacted), exports, `audit_event`, user name and email	RLS on the membership join. Full account numbers are never stored (C-34). No field-level encryption: it would break sums, ranges and reports, for a threat that volume encryption plus RLS already covers.	Members only, per §4.2.1. Payloads are manager+ only. Logs carry opaque IDs only (C-33). Exports are streamed and never persisted (C-51). Emails are visible only to co-members who are owner or manager.	Ledger: indefinite (append-only) until entity purge. Raw rows follow the ledger. `audit_event`: indefinite. On account deletion, the user row becomes a tombstone (C-52).
Operator secrets	Credential private keys, DB passwords, Plaid secret, Resend key, BLS key, OAuth client secrets, BetterAuth secret, backup key, `CRON_SECRET`	Files under `/run/secrets`, mode 0400. Never in the repo, an image or the database (§6.2).	Never logged. Prod and staging values differ. The backup key and the credential key are never stored together.	§6.4

Tier	Data	At rest (beyond volume encryption)	Exposure rules	Retention
T4 Public reference	Prices, CPI-U, tax tables, system report templates	—	No tenant column.	Indefinite

No outbound user data (a control). ARCH §1 says nothing about a user's books leaves the box, except through a provider call the user initiated or an owner-initiated export. Treat that sentence as a control. Any feature that would break it needs a security review and a PRD-level decision, not just a library import. Examples: crash reporting with payloads, hosted error tracking, telemetry that includes tenant data, or an LLM call that carries transaction text.

Stored, but never logged. T3 is stored in the clear under RLS, but scrubbed from logs. Logs get tailed over SSH, pasted into bug reports and shipped to aggregators, which makes them the likeliest escape route. That is why scrubbing is enforced by a CI canary (C-33) and not left to convention.

6. Secret Management

Two populations. Operator secrets: one set per instance, held by the person running the box. User credentials: one per connection, held sealed in the database. §6.3 resolves ARCH OQ-1.

6.1 Inventory

Secret	Lives in	Mounted into	Rotation
Credential keyring (X25519 private keys, by `key_id`)	`/etc/polycarpic/secrets/credential_keyring`, plus an offline copy in the operator's password manager	`worker` only. Public keys go to `web` as non-secret config.	§6.4
Postgres role passwords (`poly_app`, `poly_worker`, `poly_migrator`)	Host secrets directory	Only the container that uses that role	On compromise or a change of operator
BetterAuth secret	Host secrets directory	`web`	On compromise. Rotating it invalidates every session, which is the intended effect in an incident. It also encrypts TOTP seeds, backup codes and OAuth tokens (measured, 1.7.18). So rotate through BetterAuth's versioned `secrets` option: the new version encrypts, and old versions still decrypt until each row is rewritten (code-read). Replacing the secret outright would lock out every enrolled user.
OAuth IdP client secrets,	Host secrets directory	`web`	At the provider, on suspicion

Secret	Lives in	Mounted into	Rotation
Resend key			
Plaid client secret, broker OAuth clients, price-provider key, BLS key	Host secrets directory	`worker`, plus `web` for the Plaid token exchange	At the provider, on suspicion
Backup encryption key and store credential	The public half (recipient) on the box; the private half offline (§6.6)	Backup job	On a change of operator. Keep the old key until its last backup expires.
Staging credential keyring (sandbox tokens only)	The staging secrets directory, separate from prod's	Staging `worker` only	Freely: losing it means relinking sandbox items
DNS-API token (local box, real-domain mode, P7)	Host secrets directory	Caddy only	At the DNS provider, on suspicion. Scoped to one zone.
`CRON_SECRET` (Vercel only)	Vercel env var	Internal cron route	On compromise

Secret	Lives in	Mounted into	Rotation
CI deploy credential or registry token	GitHub *environment* secret, with required reviewers	Actions runner	On compromise. Revocable independently of the operator's SSH key.
Aggregator tokens and URLs (T2)	Postgres, sealed (§6.3)	Opened only in `worker`	Driven by the provider. Re-sealed on key rotation.

6.2 Operator secret handling

- **Files, not environment variables.**
  - Secrets live on the host in `/etc/polycarpic/secrets/` (root:root, files 0400, directory 0700).
  - They are mounted as compose `secrets:` at `/run/secrets/`, and apps read them through the `*_FILE` convention.
  - Why: environment variables leak through `docker inspect`, `/proc/<pid>/environ`, child processes and error reporters that dump `process.env`.
  - *Revised:* 8963efb and ARCH §9 specify a root-only env file.
- **Per-container scoping.** Each container gets only the secrets it needs. `web` never gets the credential private key or the backup key. Staging gets its own values, and its own keyring, which seals only Plaid Sandbox and SimpleFIN demo tokens. It never gets prod's keyring or the production Plaid secret (C-77). *Revised in v1.1:* staging had no keyring while it used only the fake provider.
- **Nothing in images.** No secret is ever baked into an image or passed as a build arg. `.env.example` lists names with a comment each, never values.
- **Fail closed at startup.** Outside development, the app refuses to start if a required secret is missing, or if a secret-looking env var is set in production.
- **Read once.** Secrets are read at startup. There is no runtime secret-fetch path and no admin endpoint that reveals configuration.
- **Vercel.** The tertiary target uses Vercel's encrypted env vars (R-3). The hosting order is Hetzner, then the local box, then Vercel + Neon (Principal, 2026-11-06).

- **Local box.** Same file layout as the Hetzner path. In real-domain mode (P7) the DNS-API token is a file secret mounted into Caddy only, scoped to one zone (C-76).

6.3 Credential sealing — ruling on OQ-1 (revised)

Ruling: app-layer *asymmetric* sealing in `packages/secrets`. `web` holds only public keys; the private keyring is mounted into `worker` only.

This replaces 8963efb's symmetric envelope scheme, which ARCH §3.1 and TB-4 folded in: an AES-256-GCM DEK wrapped by a master key mounted into the app. The seam is unchanged: `seal()` / `open()` with a `key_id`. Rejecting pgcrypto and external KMS still stands (§6.5).

- **Why revise.**
  - With a symmetric master key, whoever seals can also open. The sealing process is `web`, because it receives the SimpleFIN setup token or the Plaid public token.
  - `web` is also the internet-facing process, and the most likely place for RCE or SSRF. So one `web` compromise would yield every bank token (T-31).
  - Asymmetric sealing removes that path at about the same implementation cost.
- **Algorithm.** HPKE (RFC 9180) base mode: DHKEM(X25519, HKDF-SHA256), HKDF-SHA256, AES-256-GCM. The library (e.g. `@hpke/core`, or an equivalent over `node:crypto`) is pinned in the `packages/secrets` design file, with SecEng sign-off. No home-rolled construction.
- **Binding.** The HPKE `aad` is the canonical encoding of (`connection_id`, `entity_id`, `provider`, `key_id`, `alg_version`). A blob copied to another row fails to open (T-32).
- **Stored columns.** `enc` (the encapsulated key), `ciphertext`, `key_id`, `alg_version`. These replace ARCH §3.1's "ciphertext + nonce + wrapped DEK + key_id".
- **Flows.**
  - *Link* (in `web`): receive the setup or public token, exchange it with the provider, `seal()` the result under the active public key, insert the row, drop the plaintext.
  - *Poll* (in `worker`): `open()` inside one call frame, for one provider call. The plaintext is never cached and never held in a module-level variable.
  - *Unlink*: `web` enqueues a revoke job. `worker` opens the credential, calls `adapter.revoke()`, then deletes (C-32).
- **Database enforcement.** `poly_app` has column INSERT and UPDATE on the sealed columns, but no SELECT. `poly_worker` has SELECT. The columns never appear in DTOs or `openapi.json` (C-31).
- **Errors.** Provider errors are re-thrown with the credential stripped. A raw provider exception never reaches a response or a log without passing through the scrubber.

- **Import boundary.** A lint boundary rule forbids any package except `packages/secrets` from importing the crypto module. Only `apps/worker` may import `open()`.

6.4 Key rotation for an operator with no KMS

1. Run `polycarpic secrets rotate` in `worker`. It generates key *n+1*, adds it to the keyring, and publishes its public key as the active sealing key.
2. It re-seals every row with `key_id = n`: open with *n*, seal with *n+1*. The step is idempotent and resumable.
3. It asserts that zero rows remain under key *n*, removes *n*, and writes an `audit_event`.
4. When to rotate: on any suspicion that the worker, the box, the operator's SSH key, or a backup together with the keyring was compromised, and whenever the operator changes. Annual rotation is optional hygiene. If the keyring *and* the database may both be exposed, rotation alone is not enough: revoke every token at the provider and relink (RB-1).

A lost keyring means relinking, not data loss. Every connection shows "relink required". The ledger is not encrypted under this key. The runbook says this plainly, so that nobody stores the key next to the backup "to be safe". The keyring is never part of the backup set.

6.5 Why not pgcrypto, and why not a secrets manager

- **pgcrypto (rejected).** The key and the plaintext would pass through SQL. That puts them in `pg_stat_activity`, potentially in statement logs, and in the same process as the data. A stolen dump must yield inert ciphertext. With the key co-located, it does not.
- **Vault or a cloud KMS (rejected for v1).** On a single VPS, a vault's unseal material sits on the same box. So it adds a service for a novice operator to run without adding real separation. It would also break the near-zero-spend guardrail and add an availability dependency. The `key_id` seam leaves room for a KMS-backed keyring later; revisit if R-6 ever triggers.
- **The credential-free path stays the recommended default.** The CSV/OFX adapter stores no credential. An instance that never links an aggregator holds zero T2 data. The operator runbook presents linking as a convenience trade-off.

6.6 Backup key custody — owned by DevOps (ARCH OQ-9)

Requirements on INFRA:

- Backups are encrypted on the box before they leave it.
- The key can be recovered without the VPS.

- The key is distinct from the credential keyring, and the two are never stored together.

Recommended shape:

- An asymmetric recipient (age-style), so the box holds only the public half.
- A write-only or append-only store credential, or immutable snapshots.

With that shape, a fully compromised box can neither read nor delete its own historical backups. The acceptance test is a restore onto a clean machine. RPO and RTO numbers are DevOps's call.

7. Controls Catalog

Each control maps to threats in §3. No application code exists yet, so Status names the milestone where the control must land. A feature that touches a control cannot pass the Validate security gate without it. Control IDs are new in this rewrite; 8963efb's C-numbers are retired.

ID	Control, location and test	Threats	Status
C-01	Invite-only registration after the bootstrap user. OAuth sign-in for an unknown email is rejected unless an invite is pending. There is no reopen switch (P6). The operator CLI `polycarpic user invite <email>` issues an audited invitation that shares no entity. **Lives at:** `packages/auth` hooks; worker CLI **Tested by:** Integration: • a 2nd signup is rejected; • OAuth for an unknown email is rejected; • an invite accept works; • a CLI invite yields a user with only their own Person, and writes an `audit_event`; • a config scan finds no self-signup flag.	T-01	POLY-M1
C-02	Session cookie is `__Host-`, `HttpOnly`, `Secure`, `SameSite=Lax`. 7 days idle, 30 days absolute. All sessions revoked on a password or MFA change. **Lives at:** `packages/auth` config **Tested by:** Playwright `Set-Cookie` assertions; config unit test.	T-04	POLY-M1
C-03	MFA (TOTP, passkeys) per §4.1: required for owner and manager, except the owner row on a user's own unshared Person (OQ-34); optional for viewers. Step-up re-auth within 5 minutes on the listed actions, always with the second factor. Operator MFA-reset CLI, audited. A role raised to manager or owner is held in `membership.pending_role`, with `role` unchanged, until the member enrols; only the `poly_govern` promotion makes it effective (§4.1; ARCH §3.1c). The first factor is enrolled only after the password and a code sent to the account's verified email are confirmed. The code is	T-02, T-08, T-81, T-82, T-83	POLY-M1 (MFA, required for owner and manager per P1), then with each gated feature

ID	Control, location and test	Threats	Status
	single-use, lasts 10 minutes, is voided after 5 wrong entries, is sent at most 3 times per 15 minutes, and is bound to the session; BetterAuth's enrolment endpoints are wrapped by a `hooks.before`, and its `otp` method is off (§4.1). BetterAuth's writes run inside the request's `withTenant` transaction through the routing pool (A-31), so the database guard checks the grant by user and session. A completed first enrolment sets the step-up stamp on the session that BetterAuth issues to replace the old one (A-32). A backup code counts as step-up only once `second_step_at` is set. **Lives at:** `packages/auth`; the `requireRecentAuth()` guard; worker CLI; `packages/tenancy`; the ARCH §3.1c triggers **Tested by:** Integration: every step-up action returns 403 without a recent re-auth; the CLI writes an `audit_event`. pgTAP: a viewer raised to owner without a second step is refused a manager write and an owner write, is not counted by `app.owner_count`, and is not a required approver; after enrolment all four flip and an `audit_event` is written. pgTAP: `poly_app` cannot set `second_step_at` or promote a held row; at 1 → 2 a direct-path held raise is cancelled and an approved one kept; removing the last method is refused while manager and allowed while viewer. OQ-34, integration: an unenrolled user whose only membership is their own unshared Person can sign in and view; their first gated action (creating a Trust, the first invitation on the own Person, linking it under a parent, an export, a bank link) redirects to Set up a second step and is not performed, with no row written; each succeeds after enrolment. Email anchor, integration: a session with only the password cannot complete a first enrolment (no factor row, `second_step_at` null) through the UI or by calling `/two-factor/enable`, `/two-factor/verify-totp`, `/passkey/generate-register-options` or `/passkey/verify-registration` directly; a backup code from an unverified enrolment does not pass step-up; a first TOTP verify that the guard refuses leaves `twoFactorEnabled` false and the old session in place (one transaction); an unenrolled user's gated action (a Trust, an export) is not performed until both the password and the emailed code are confirmed and the factor is set; a code from session A is refused in session B; a used, expired or 5-times-wrong code is refused; the 4th send in 15 minutes is suppressed with an identical response; the code		

ID	Control, location and test	Threats	Status
	goes to the current address during a pending email change; a user who already has a factor adds a second without any email code. Step-up: after a completed first enrolment the gated action proceeds with no second Confirm it's you, and a 403 follows after 5 minutes. pgTAP (A-29): `poly_app` cannot insert a passkey, set `twoFactorEnabled`, or mark TOTP verified for a user with no factor and no live grant for the session in `app.session_id`. Integration (A-31): the routing pool is tested against the pinned BetterAuth version. A write outside `withTenant` fails closed, and a rollback removes BetterAuth's rows.		
C-04	BetterAuth `rateLimit` with database storage (e.g. sign-in 5/min per IP and per email). Generic auth error messages. Body-size limits on every route. **Lives at:** `packages/auth`; `hooks.server.ts` **Tested by:** Integration: the 6th attempt returns 429; responses for known and unknown emails are identical.	T-02, T-07, T-09	POLY-M1
C-05	OAuth uses PKCE and `state`. Accounts link only when the IdP asserts a verified email *and* there is either an existing session or a matching pending invite. The providers are Google, Apple and GitHub (Principal, 2026-11-08), and each asserts verification differently: • **Google:** the ID token's `email_verified` is true. • **Apple:** the ID token's `email_verified` is true, whether sent as a boolean or the string `"true"`. A private-relay address is a distinct email: it links only through an existing session. • **GitHub:** the profile carries no verification claim. The email is taken only from the `/user/emails` entry that is both `primary` and `verified`. The public profile email is never used. The requested scopes are `read:user` and `user:email`, nothing more. **Lives at:** `packages/auth` **Tested by:** Integration: a mocked IdP sending `email_verified=false` does not link, for each of the three; Apple's string `"false"` does not link; a GitHub account whose matching email is unverified, or verified but not primary, does	T-03	POLY-M1

ID	Control, location and test	Threats	Status
	not link; the GitHub authorize request asks for no scope beyond the two.		
C-06	`kit.csrf.checkOrigin = true`, never disabled. Mutating `/api/v1` routes accept JSON only. SameSite cookies. **Lives at:** `svelte.config.js`; `/api/v1` wrapper **Tested by:** Integration: a cross-origin form POST is rejected; `text/plain` returns 415.	T-05	POLY-M1
C-07	CSP through `kit.csp`: nonce or hash, no `unsafe-inline` script, `frame-ancestors 'none'`, `object-src 'none'`. HSTS for 1 year at Caddy. `nosniff`. `Referrer-Policy: strict-origin-when-cross-origin`. **Lives at:** `svelte.config.js`; Caddyfile **Tested by:** Header assertions in the Playwright smoke run.	T-04, T-06	POLY-M1
C-10	Role separation and hygiene per §4.3(k). The hygiene set is every NOLOGIN definer-owning role: `poly_authz`, `poly_purge`, `poly_rollup`, `poly_graph`, `poly_stake`, `poly_govern` and `poly_journal`. Each is NOBYPASSRLS, owns no tables, and is a member of no role and granted to none. No TRUNCATE, REFERENCES or TRIGGER grants to login roles. No CREATE on `public`. **Lives at:** `packages/db` role migrations **Tested by:** pgTAP catalog assertions.	T-13, T-21	POLY-M1
C-11	`withTenant()`: `set_config('app.user_id', …, true)` as the first statement; branded `SessionUserId`; nesting guard; raw pool not exported; session-level `SET` banned by lint; pooler-mode assertion in the deploy checklist. **Lives at:** `packages/db`; INFRA checklist **Tested by:** pgTAP pool-reuse test (tenant A, then tenant B on the same backend, sees 0 of A's rows); Vitest: a nested call throws.	T-11	POLY-M1
C-12	Fail-closed context: `app.current_user_id() = nullif(current_setting('app.user_id', true),`	T-11	POLY-M1

ID	Control, location and test	Threats	Status
	`''),'')::uuid`. **Lives at:** `packages/db` SQL **Tested by:** pgTAP: on a backend that already carried a context, a query with no context returns 0 rows on every tenant table and raises no error.		
C-13	ENABLE + FORCE RLS and at least one policy on every table with a tenant column. The probe is catalog-derived, with an inversion canary. The global-table allowlist is a security-gate decision. **Lives at:** `packages/db` battery **Tested by:** pgTAP in CI; the build fails if any table is uncovered.	T-10, T-12	POLY-M1
C-14	Per-command policies with `WITH CHECK`; no `FOR ALL`. Tenancy and actor columns are not updatable. No DELETE policies on append-only tables. The entity-type exception goes through a label-only `security_invoker` view. **Lives at:** `packages/db` schema **Tested by:** pgTAP: a viewer INSERT is denied; an insert with a forged `entity_id` is denied; an `UPDATE` of `entity_id` gets permission denied.	T-10, T-14, T-19	POLY-M1
C-15	Authz functions are owned by `poly_authz`, with a role-targeted policy on `membership`. Every definer function sets `search_path = ''` and has EXECUTE revoked from PUBLIC. SECURITY DEFINER functions are allowlisted. The allowlist includes `poly_journal`'s numbering and chain trigger functions (C-24). **Lives at:** `packages/db` **Tested by:** pgTAP: the set of `prosecdef` functions equals the allowlist; `proconfig` pins `search_path`; a manager can list members without recursion.	T-13, T-20	POLY-M1
C-16	Views are `security_invoker = true`. No materialized views over tenant data. **Lives at:** `packages/db` **Tested by:** pgTAP catalog check on `reloptions` and `relkind`.	T-13	POLY-M1

ID	Control, location and test	Threats	Status
C-17	The §4.2.1 matrix is stored as data. QA generates an endpoint × role test matrix, and a route with no matrix row fails CI. **Lives at:** `packages/tenancy`; QA suite **Tested by:** Generated Vitest + pgTAP tests; coverage check.	T-10, T-25	POLY-M1, growing with each feature
C-18	Tenant-visible primary keys are UUIDs. No sequential IDs in URLs or the API. **Lives at:** `packages/db` schema **Tested by:** Schema test: every tenant PK has type `uuid`.	T-17	POLY-M1
C-19	Parent links (amended in v1.1, OQ-19): • **Insert** needs owner on both the child and the parent. This is unchanged, and it is what stops T-16. • **Delete** needs owner on either end. • **Read.** A link is readable with any membership on the parent, or owner on the child. `app.parents_of(entity_id)` returns ancestor ids and display names, with the direct parents marked, to the child's owner only. • **No `poly_app` or `poly_worker` tenant policy reads `entity_rollup`.** Structure grants no row. The only policies that consult it are the role-targeted ones of `poly_rollup` (C-25). **Lives at:** `packages/tenancy`; `entity_rollup` policies **Tested by:** pgTAP: • owner of the child only is denied the insert; • a member of the parent sees no child rows; • a non-member reads no link; • `parents_of` returns names only, and nothing to a non-owner; • a catalog check that no policy for `poly_app` or `poly_worker` references `entity_rollup`.	T-16, T-71	POLY-M1 (junction); `parents_of` with POLY-1.1

ID	Control, location and test	Threats	Status
C-25	Roll-up and graph definers (OQ-19, OQ-25): • **`poly_rollup`** owns `app.rollup_totals`, `app.rollup_economic`, `app.rollup_tree` and `app.parents_of`. ◦ NOLOGIN, NOBYPASSRLS, owns no tables, SELECT only. ◦ Its role-targeted policies are confined to `app.rollup_descendants()` (plus the equity rows of one-link entity holders of those entities, for weighting), and, for `parents_of`, to the ancestors of entities the caller owns. ◦ Results come back through a fixed `RETURNS TABLE` of aggregates by entity, report line and currency: never an account, entry, line, payee, memo, lot or position. ◦ The guard is viewer+ on the parent. ◦ `rollup_economic` takes no Person argument. • **`poly_graph`** (new) owns the BEFORE INSERT trigger functions `app.check_rollup_edge()` and `app.check_equity_holding()`. ◦ It has graph-wide SELECT on `entity_rollup` and `equity_holding`, SELECT on owner `membership` rows and `entity.kind`, and INSERT on `notification`. ◦ The wide read is acceptable because a function returning `trigger` cannot be called to return data. ◦ Refusals name no entity (R-15). ◦ Both triggers take the advisory lock. • **Placement notice.** A notice goes to every owner beneath, in the same transaction, with an `audit_event`. It names the new top-level parent and says what becomes visible. • **Ancestor-invite notice (P5, 2026-11-07).** When an invitation to a parent becomes live (after any §4.2.2	T-70, T-71, T-72, T-73, T-77, T-78	Triggers with the junction (POLY-M1); definers and popover with POLY-1.1

approval, before the invitee's first sign-in), every owner of every entity beneath that parent, at any depth, gets a notification. It is written in the same transaction by a `poly_graph` trigger (AFTER INSERT on `invitation`, and on `membership` for a membership created without one, deduplicated per person).

  - It names the parent and the new member, and carries the consent-disclosure sentence and the pull-out link.
  - Any role triggers it, because viewers see totals too.
  - A later role raise on the parent does not.
  - Nothing waits on the recipients.

- **Consent disclosure (P2, 2026-11-07; text fixed in v1.2).** The inclusion step (adding an entity to a parent), the placement notice and the new-member-above notice carry this text verbatim, as three sentences, with `<parent>` the only variable:

  > Members of <parent>, and of any parent above it, see class and category totals for this entity for any period.
  > For a small entity, that effectively reveals individual amounts and their dates.
  > Nothing else is shown: no payee, memo, account or counterparty.

  - Sentences two and three confirm the DESIGN v0.16 rendering. Sentence one is amended: "Members of <parent> see..." understated the audience. A member of any ancestor can call the roll-up on that ancestor, and its descendants, at any depth, include this entity (`app.rollup_descendants()`). So members of every parent above see the same totals, now and after any later placement (T-73).

ID	Control, location and test	Threats	Status
	◦ `<parent>` is the parent the entity is being added to, or, in a notice, the parent named in its first sentence. The wording is a security requirement. The Playwright check compares the rendered block to this text exactly. Changing it needs a security-gate review. Any date range is allowed: there is no period guard. • **Holdings popover on a parent.** Positions come only through ordinary `poly_app` RLS, for the parent's own accounts and for entities where the viewer is a member. Any other entity shows its total and the boundary panel. No definer returns a position. • All of these functions are on the definer allowlist (C-15) and in the role-hygiene set (C-10). **Lives at:** `packages/db` SQL; `packages/tenancy` **Tested by:** pgTAP (ARCH §3.3a–c tests, plus): • a parent viewer without child membership gets totals equal to the child owner's figures, and 0 rows from every tenant table and from the popover's position read for that child; • a cycle through an entity invisible to both the inserter and `poly_rollup` is refused; • Playwright: the inclusion step, the placement notice and the ancestor-invite notice render the consent-disclosure text; • pgTAP: an invitation going live on a Holding notifies the owners of a Person two levels down whom the inviter cannot see, and a role raise does not; • calling a `poly_graph` function directly fails; • the result-column set of each `poly_rollup` function equals its pinned shape.		
C-27	Cap table and allocated income, under the new `poly_stake` role (NOLOGIN, NOBYPASSRLS, owns no tables, SELECT only; OQ-25):	T-74, T-75	POLY-1.0 (tax allocation)

ID	Control, location and test	Threats	Status
	• **`app.cap_table_candidates(entity_id)`** is for an owner of `entity_id` only. It returns the id and display name of members' own Persons, and of Person, Trust and Business entities visible to the caller. Nothing else. • **`app.my_stakes()`** is a holder's owners' read path. It returns entity id, display name, units, total units and effective-from, for holder entities the caller owns (or the caller's own Person). Other holders' rows are never returned. The `equity_*` SELECT policies are not widened for non-members. • **`app.allocated_income(taxpayer_entity_id, from, to)`** returns income totals by tax tag × stake, never rows. ◦ Its reads are confined by role-targeted policies to `app.held_entities()`: entities in which the caller's own Person, or an entity the caller owns, has ever held units, plus one link onward. ◦ Guard: the caller has a membership on the taxpayer entity. • **Holder notification** is written by the `poly_graph` trigger (C-25). **Lives at:** `packages/db` SQL; `packages/tax` **Tested by:** pgTAP (ARCH §3.1a tests, plus): • a non-owner calling `cap_table_candidates` gets 0 rows; • `my_stakes` returns only the caller's holder rows; • a holder's owner reads 0 rows from `equity_holding` directly on a held entity where they are not a member; • `allocated_income` returns totals equal to Σ(tagged income × stake), and 0 rows for an entity outside `held_entities()`.		; `my_stakes` with the cap-table editor
C-20	Only owners change membership. A trigger keeps at least one owner (except during purge). Every change	T-15	POLY-M1

ID	Control, location and test	Threats	Status
	writes an `audit_event`. With more than one owner, the actions in §4.2.2 go through C-28. **Lives at:** `packages/tenancy`; trigger **Tested by:** pgTAP: a manager's grant is denied; removing the last owner raises.		
C-28	Co-owner approval (§4.2.2, P5b): • `change_request` and `change_approval` tables. • Direct `poly_app` writes for (a) to (c) pass only while `app.owner_count = 1`. • `app.apply_change` is owned by `poly_govern`. It recomputes the approvers at apply time and re-validates the payload. • Requests expire after 14 days. Step-up on request, approve and decline. An audit event at every step. • The invite email is sent only on apply. • At the 1 → 2 owner transition, every outstanding direct-path invitation is revoked in the same transaction, under a lock on the entity row (OQ-31, §4.2.2). • The founding owner, with `app.force_change`, can force only co-owner removal or demotion, after a decline or expiry. Transfer is founder-only, and the founder must hold owner. **Lives at:** `packages/tenancy`; `packages/db` SQL **Tested by:** pgTAP: • with 2 owners, a direct membership INSERT, a role raise or an `equity_holding` INSERT by one owner is denied; • `apply_change` with one approval missing is denied; • an approval by an ex-owner does not count; • a decline or an expiry blocks; • a stale cap-table payload is declined; • a sole owner's invite applies at once; • removing a viewer stays single-owner;	T-26, T-15, T-81	The gate, the founder trigger and the 1 → 2 revocation in POLY-M1, with tenancy (ARCH §3.1b); the request flow in POLY-1.0 (sharing, POLY-70); the cap-table edit with the editor

ID	Control, location and test	Threats	Status
	• `force_change` by a non-founder, on a pending request, on an invite or a cap-table edit, or targeting the founder, is denied; • a forced removal notifies the target and writes `change.forced`; • the founder cannot drop their owner role without first transferring the founder role; • the tombstone is refused while the user is still the founder of a multi-owner entity; • when a second owner joins, every other outstanding direct invitation (any role) is revoked in that transaction and its inviter notified; an invitation applied through `apply_change` is not; • a revoked invitation's token is refused at acceptance; • two direct owner invitations accepted concurrently end with exactly two owners and the other invitation revoked; • after dropping back to one owner, a new direct invitation is revoked again at the next 1 → 2 transition; • a raised owner still waiting for enrolment does not trigger the transition; enrolment does. Integration: no invite email before apply; each step without a recent re-auth returns 403.		
C-21	Append-only: row-level BEFORE UPDATE/DELETE triggers plus a statement-level BEFORE TRUNCATE trigger. The only exception is `current_user = 'poly_purge'` together with a matching `app.purge_entity`. **Lives at:** `packages/db` SQL **Tested by:** pgTAP: UPDATE, DELETE and TRUNCATE all raise for `poly_app`, `poly_worker` and `poly_owner`; `poly_app` with the GUC set still raises.	T-21, T-50	POLY-M2
C-22	Deferred balance constraint trigger (ARCH §3.2).	T-22	POLY-M2

ID	Control, location and test	Threats	Status
	Lives at: `packages/ledger` SQL **Tested by:** pgTAP + property-based tests.		
C-23	A BEFORE INSERT trigger *forces* the actor from the session context. `actor_kind = user` requires `actor_id = app.current_user_id()`. **Lives at:** `packages/db` SQL **Tested by:** pgTAP: a forged `actor_id` is overwritten or rejected.	T-23, T-25	POLY-M2
C-24	Per-entity ledger hash chain (Principal, 2026-11-07, P4): • **Hash.** `entry_hash = sha256(prev_hash ‖ canonical(entry, its lines))`, in `journal_number` order per entity. The canonical encoding is pinned and versioned: field order, numeric text form and UTF-8 NFC. • **When it is written.** The hash covers the lines, so it is written at commit by a deferred constraint trigger, after the balance check. It goes into an append-only `journal_chain` row (entity, `journal_number`, entry, `prev_hash`, `entry_hash`, `alg_version`). The posted entry is never UPDATEd. The chain head sits on `entity_journal_counter`, whose row lock already serializes an entity's postings. • **Who writes it (OQ-30, confirmed with changes).** `poly_app` and `poly_worker` cannot write `journal_chain` or `entity_journal_counter`. The numbering and chain triggers are `SECURITY DEFINER` functions owned by `poly_journal`, NOLOGIN and NOBYPASSRLS. Its grants, each through a role-targeted policy (`TO poly_journal`), are: ◦ SELECT, plus UPDATE of `last_number`, `head_number` and `head_hash` only, on `entity_journal_counter`; ◦ INSERT on the counter, used only by an AFTER INSERT trigger on `entity` that creates the row with number 0 and the fixed seed. No other role	T-24, T-21, T-80	POLY-M2 (with the ledger); the numbering trigger with POLY-64

ID	Control, location and test	Threats	Status

inserts a counter row, so no caller can pre-seed a head;

- INSERT on `journal_chain`, never UPDATE or DELETE. `journal_chain` is append-only under C-21, and only the purge deletes it;
- SELECT on `journal_entry`, `journal_line` and `journal_chain`. The read is graph-wide, which is acceptable for the reason given for `poly_graph`: a function returning `trigger` cannot be called to return data.

- **No line after the chain row.** `SET CONSTRAINTS … IMMEDIATE` is open to any role, so `poly_app` could fire the deferred chain trigger before an entry's lines exist. A BEFORE INSERT trigger on `journal_line`, also owned by `poly_journal`, refuses a line whose entry already has a `journal_chain` row. The chain therefore always covers every line, and an early fire fails the posting instead of leaving a false SEV1 for the verifier.
- **Readers.** `poly_app` and `poly_worker` read `journal_chain` and the head through ordinary tenant RLS, viewer and up, for the export head and the verifier. The verifier writes only `audit_event` and `notification`.
- **Verifier.** A weekly job recomputes each entity's chain. A break writes an `audit_event` and emails the entity's owners and the operator, as SEV1 ledger tampering (§10).
- **Exports.** Every export carries the chain head (`journal_number`, `entry_hash`), and the `export.completed` audit event records it.
- **Purge.** A purge deletes the entity's chain, and the `entity.purged` event records the final head.
- **Limits.** A superuser can recompute the whole chain. Detection therefore rests on a head held off the box: an export the owner keeps (R-1).

ID	Control, location and test	Threats	Status
	Lives at: `packages/ledger` SQL; worker verifier job; export module **Tested by:** pgTAP and integration: • an edit made as superuser (triggers disabled) to an entry, a line or the order is detected by the verifier; • a rolled-back posting leaves no chain row and no gap; • `poly_app` cannot insert or update `journal_chain`, or insert or update an `entity_journal_counter` row; • a new entity gets its counter row at number 0 with the seed, and a caller-supplied `journal_number` is overwritten; • with `SET CONSTRAINTS ALL IMMEDIATE`, inserting an entry then its lines is refused, and no chain row commits; • catalog: each `poly_journal` function is attached as a trigger only to its intended table (`pg_trigger.tgfoid` to `tgrelid`), and a direct call fails; • an export's head matches the verifier's; • a correction (reversal plus replacement) takes two chain links.		
C-30	HPKE sealing with AAD binding. `web` seals, `worker` opens; keys are held in a keyring by `key_id` (§6.3). **Lives at:** `packages/secrets` **Tested by:** Unit: round trip works; a swapped AAD, tampered ciphertext or unknown `key_id` all fail closed. Integration: the `web` container has no keyring file.	T-30, T-31, T-32	Seam in POLY-M2; first real use in POLY-1.0 (POLY-77)
C-31	`poly_app` has no SELECT on the sealed columns. DTOs and the OpenAPI spec omit them. **Lives at:** `packages/db`; DTOs **Tested by:** pgTAP: permission denied. OpenAPI freshness diff.	T-31, T-33	POLY-M2
C-32	Unlink enqueues a worker revoke job. The row stays `revoke_pending` until the provider confirms, with retries	T-34	POLY-M2 (fake

ID	Control, location and test	Threats	Status
	and backoff; a failure is surfaced to the user. Every step writes an `audit_event`. **Lives at:** `packages/import`; worker **Tested by:** Integration: when the fake provider's revoke returns 500, the row is not deleted.		provider), POLY-1.0
C-33	Logs are safe by construction. Pino serializers log only declared fields, and `redact` covers known paths. Never logged: bodies, `Cookie`, `Authorization`, SQL parameters, memo, payee. **Lives at:** Shared logger package **Tested by:** **Canary test:** the integration run seeds canary token, URL, account-number and memo values. CI captures every log line and fails if any canary appears.	T-33	POLY-M1 (harness), extended with each adapter
C-34	Adapters redact full account numbers, PANs and credential-bearing fields from stored payloads, keeping last-4 only. Payloads are readable by manager+ only. **Lives at:** Each `packages/import-*` **Tested by:** Per-adapter fixture containing canary numbers; pgTAP: a viewer cannot read the payload.	T-33, T-36	POLY-M2, POLY-1.0
C-35	`polycarpic secrets rotate` (§6.4). **Lives at:** `packages/secrets` CLI **Tested by:** Integration: after rotation every row opens under the new key and none under the old.	T-30, T-31	POLY-1.0
C-40	Upload caps (e.g. 10 MB, 50k rows). OFX 2.x parsed with DTD and external entities disabled; OFX 1.x parsed with a non-XML parser. No eval. **Lives at:** File adapter **Tested by:** Unit: XXE and billion-laughs fixtures are rejected; an oversize file returns 413.	T-35, T-09	With the file adapter (POLY-77)
C-41	Svelte escaping. `{@html}` is lint-banned everywhere except one sanitizing component. CSV export neutralizes any cell starting with `= + - @`, tab or CR. **Lives at:** `packages/ui` lint; export module **Tested by:** Unit: a `=HYPERLINK(…)` cell exports inert. Playwright: a `<script>` payee renders as text.	T-06, T-55	POLY-M1 (lint), then with the export feature

ID	Control, location and test	Threats	Status
C-42	The report formula engine parses formulas to an AST over an account allowlist. `eval` and `new Function` are banned. Node-count, depth and time caps apply. **Lives at:** `packages/reports` **Tested by:** Unit: pathological and malicious formulas are rejected.	T-09	POLY-M5
C-43	Zod validates every route and form action. Unknown keys are stripped; a mismatch returns 400. **Lives at:** `apps/web` wrapper **Tested by:** Unit tests on the wrapper.	T-10, T-14	POLY-M1
C-44	Parameterized queries only. Lint bans `sql.raw()` and `sql.unsafe` with non-literal arguments. Raw SQL is allowed only in reviewed migrations. CodeQL `js/sql-injection` runs in CI. **Lives at:** ESLint; CI **Tested by:** Lint + CodeQL.	T-18	POLY-M1
C-45	Reference-data anomaly flag: a price move over 50% day-over-day, or an out-of-bounds CPI revision, is stored but flagged. **Lives at:** `packages/refdata` **Tested by:** Unit.	T-43	POLY-M4
C-46	Attachments (ARCH §3.5 @ fcffe6c): • Stored as `bytea` under RLS. The bytes live in `attachment_blob`, separate from the metadata. • Size limit of 10 MB. Type is sniffed server-side against an allowlist; the client-declared type is never trusted. • Served only as a download: `Content-Disposition: attachment` with an RFC 6266 sanitized `filename*`, plus `nosniff` and a CSP of `default-src 'none'; sandbox`. • Composite entity FKs on attachments and their links. • Insert-only, and included in the purge. • Per-entity storage quota and upload rate limit.	T-37	With the attachments feature

ID	Control, location and test	Threats	Status
	• File contents are never parsed. **Lives at:** `packages/attachments` (per ARCH) **Tested by:** Integration: a spoofed type is rejected; a file name containing CR/LF is sanitized; response header assertions; a cross-entity link is denied; purge leaves 0 attachment rows.		
C-50	Purge per §4.5: the `poly_purge` role, 8 guards checked in order, and a typed confirmation. **Lives at:** `app.purge_entity` ; tenancy route **Tested by:** pgTAP: a non-sole owner is denied; a stale export is denied; a write after the export is denied; `poly_app` with the GUC set is denied outside the function; a successful purge leaves 0 rows in every purgeable table plus 1 `audit_event`.	T-50, T-51, T-08	The purge feature (POLY-1.0 scope)
C-51	Export is streamed, never persisted. If it has to run async, the file is encrypted in temp storage and expires after 24 h. Owner only, with step-up; audited with a SHA-256; rate-limited. **Lives at:** Export module **Tested by:** Integration: no file remains after download.	T-53	The export feature
C-52	Account deletion: the user row becomes a tombstone; ledger rows are never UPDATEd. BetterAuth `deleteUser` is overridden. Sessions, accounts, MFA and passkeys are deleted. Linked credentials are revoked, and the user's connections go `principal_invalid` . *Revised in v1.1 (ARCH §3.4a):* each solely-owned entity is resolved in the flow by transfer to a named current member, or by a purge through C-50, or the user aborts. The deleting user is the only invoker (§4.6). The tombstone transaction re-checks that no sole-owned entity remains. **Lives at:** `packages/auth` override; tenancy **Tested by:** Integration: • entries remain and resolve to "deleted user"; • the §4.6 negatives; • abort before the final confirmation changes nothing.	T-41, T-54, T-84	POLY-1.0

ID	Control, location and test	Threats	Status
C-53	`audit_event` is append-only, has no purge exception and no FK to `entity`. Readable by owners and by the acting user. **Lives at:** `packages/db` **Tested by:** pgTAP: the row survives a purge and cannot be updated.	T-52, T-25	POLY-M1 (the table), then events with each feature
C-60	Job principal per §4.4: `linked_by_user_id`, `principal_invalid`, no context-free fallback, `app.job_targets()`, role-targeted reference-data policies, restricted `poly_worker` grants. **Lives at:** `packages/db`; `apps/worker` **Tested by:** pgTAP: a job whose linker was revoked writes 0 rows and flips the connection's state; `poly_app` cannot write reference data.	T-40, T-41	POLY-M2 (POLY-64)
C-61	Vercel topology (OQ-13): two Vercel projects from the same repo. • `web` exposes only user routes. It holds only the public keys and the `poly_app` DB credentials. • `jobs` exposes only cron job routes. Each route checks `CRON_SECRET` with `timingSafeEqual`. It holds the keyring and the `poly_worker` credentials. • Job routes are absent from the node build and from the `web` project. **Lives at:** `apps/web`, `apps/worker` (Vercel entry) **Tested by:** Build tests: • the route manifest of each project equals its allowlist; • the `web` build contains no keyring reference; • the node build returns 404 for job routes. Env check in CI: `web` has no keyring variable.	T-42, T-31	POLY-M1 (split); keyring before POLY-77
C-70	Only Caddy publishes ports. Prod and staging use separate compose networks. No Docker socket mounts.	T-60, T-65	POLY-M1 (INFRA)

ID	Control, location and test	Threats	Status
	Containers run non-root, `read_only`, with `cap_drop: [ALL]` and `no-new-privileges`. **Lives at:** `compose.yml` (DevOps) **Tested by:** Compose policy lint in CI; a post-deploy external port scan shows only 22, 80 and 443.		
C-71	SSH is key-only, with no root login. Unattended security upgrades. The firewall is the Hetzner Cloud Firewall, which sits outside the host, so Docker cannot bypass it. This covers the Hetzner target; the local box uses C-76 instead. **Lives at:** INFRA provisioning **Tested by:** Provisioning check script.	T-60	POLY-M1 (INFRA)
C-72	The Postgres volume and local backup staging sit on an encrypted (LUKS) volume. **Local box: unlocked automatically by the TPM** (Principal, 2026-11-07, P8): • LUKS2, with a TPM2 key slot (`systemd-cryptenroll --tpm2-pcrs=7`, plus PCR 11 where the distro ships a unified kernel image; unmeasured per distro), sealed to the Secure Boot state. Booting any other OS or kernel changes the PCRs, and the disk stays locked. • A second, recovery passphrase key slot, kept offline in the operator's password manager, never on the box. • Firmware hardening: ◦ Secure Boot on; ◦ a firmware setup password; ◦ boot from USB or network disabled. • Console hardening: ◦ no console autologin; ◦ the initrd emergency shell and single-user mode require the root password, so an unlocked disk is never handed to a console without authentication. **Lives at:** INFRA **Tested by:** Runbook verification.	T-61, T-69	INFRA v1; the local-box variant before the first home-box deploy

ID	Control, location and test	Threats	Status
C-73	Backups per §6.6: asymmetric recipient, a key separate from the keyring, an append-only or immutable store, and a monthly restore drill. **Local box (Principal, 2026-11-07, P9):** a nightly backup, encrypted on the box, to an append-only internet storage box (a Hetzner Storage Box or equivalent). The box's credential can add backups but never delete or overwrite them. A rotated removable drive is optional, never required, and is never kept beside the box. The off-box restore drill applies unchanged. **Lives at:** ARCH §7.2 @ 6b2f11b: nightly `pg_dump` plus WAL, each encrypted to an age recipient; Hetzner Storage Box with an append-only subaccount, or S3 Object Lock **Tested by:** A monthly restore drill, run *off the box* because the private key never touches the VPS; a test that the store credential cannot delete.	T-61, T-62	Specified in ARCH; lands in POLY-M1 (INFRA); the local-box variant before the first home-box deploy
C-74	Secrets are files, scoped per container (§6.2). Staging has its own distinct values and its own keyring (C-77). The app refuses to start if a secret is missing. No secrets in images. **Lives at:** INFRA; config loader **Tested by:** Trivy secret scan; config-loader unit test.	T-66, T-31	POLY-M1
C-75	Postgres logging: `log_parameter_max_length = 0`, `log_parameter_max_length_on_error = 0`, `log_statement = 'ddl'`. **Lives at:** Postgres config **Tested by:** Config assertion in the battery.	T-33	POLY-M1
C-76	Local-network box posture (OQ-24, ARCH §7.8). This replaces C-71's cloud firewall on that target: • **Router.** No port forwarded, and UPnP/NAT-PMP disabled on the router. • **IPv6.** The box's IPv6 is disabled, or the host firewall drops all inbound IPv6. A home router may pass inbound IPv6 to a global address. • **Host firewall.** nftables, default-deny inbound. Allowed in: 443 (and an 80 redirect) and SSH from the LAN CIDR only.	T-60, T-67, T-68, T-79	Before the first local-box deploy

ID	Control, location and test	Threats	Status

- **Ports.** Caddy is the only published port, bound to the LAN address (`"<lan-ip>:443:443"`). A `DOCKER-USER` chain rule drops non-LAN sources, because Docker-published ports bypass ufw.
- **Outbound.** Limited to DNS, NTP and HTTPS (443).
- **SSH.** Key-only, no root login.
- **Updates.** Unattended security upgrades.
- **Monitoring.** An outbound heartbeat, and its absence alerts.
- **Remote access.** None from outside the LAN in v1. A VPN later is a new security review.
- **TLS: two modes** (Principal, 2026-11-07, P7). Plain-http sign-in is refused in both: Caddy redirects 80 to 443, and the app rejects any auth route reached over plain http, trusting the forwarded protocol only from Caddy. `__Host-` cookies need `Secure` anyway.
  - **Default, real-domain mode.**
    - A subdomain of a domain the operator owns, resolved only on the LAN (split-horizon). There is no public A or AAAA record.
    - A DNS-01 wildcard certificate (`*.home.<domain>`), so the box's own name stays out of public CT logs.
    - The registrar or DNS-API token is limited to that one zone, a file secret mounted into Caddy only (R-18).
    - Google, Apple and GitHub sign-in work.
  - **Supported fallback, `.local` with a private CA.**
    - The operator runs a private CA and installs its root on each device. The root carries X.509 `nameConstraints` permitting only the box's name. Browser enforcement of constraints on a locally installed root is unmeasured. Measuring it, for each supported browser, is a **gate item for the first home-box deploy** (R-20).

ID	Control, location and test	Threats	Status

    - The root key is generated off the box and kept offline (the operator's password manager). The box holds only a name-constrained intermediate that issues short-lived leaf certificates (T-79, R-20).
    - In private-CA mode the Google, Apple and GitHub buttons are hidden *and* every OAuth route returns 404 server-side. Sign-in is email + password + the second step (P1).
    - Whether passkeys work on a `.local` RP ID is unmeasured; TOTP is always offered.
  - **The mode is explicit, and checked at startup (OQ-32, amended in v1.2).** `TLS_MODE` must be set; nothing infers it. The check tests the zone, never the host, because in real-domain mode the host deliberately has no public record:
    - **Offline, decisive.** A public origin ending in `.local`, or whose registrable domain is not under a public suffix (the bundled Public Suffix List), requires `private-ca`.
    - **Online.** Any other name requires `real-domain`, and its registrable domain's zone must resolve publicly (an SOA from a public resolver). An NXDOMAIN for the zone refuses start. A DNS timeout does not: the app starts, logs the unverified check, and retries it. A missed check costs only sign-in and certificate issuance, never exposure. A public A or AAAA record for the box's own name is logged as a warning, because it breaks the split-horizon promise.
    - A mismatch either way refuses start, with the reason logged.
    - **Certificate issuer.** The provisioning check and the heartbeat health check confirm the served leaf matches the mode: in private-CA mode it chains to the box's name-constrained

ID	Control, location and test	Threats	Status

intermediate, and in real-domain mode to a public ACME issuer. A mismatch alerts the operator.

- **Moving a `.local` box to a real domain later.**
  - Every session ends, because cookies are host-bound.
  - Passkeys do not survive, because they are bound to the old RP ID. Before the move, the runbook requires each owner and manager to hold TOTP or backup codes, with the operator's `reset-mfa` CLI as the fallback.
  - Google, Apple or GitHub sign-in is then added by linking from Settings while signed in (C-05: an existing session and a verified email). It is never auto-linked by email.
  - Every device should remove the old private root.

Lives at: INFRA local-box provisioning (DevOps)
Tested by:

- a provisioning check script (nftables rules, the `DOCKER-USER` rule, the publish binding, the sshd config);
- an outside-in scan, from a host off the LAN, of the home's public IPv4 and the box's IPv6 address, showing nothing open;
- a test that an outbound connection to a non-allowed port fails;
- an auth route over plain http is refused in both modes;
- in private-CA mode every OAuth route (all three providers) returns 404;
- startup: a `.local` name, or a name under no public suffix, outside private-CA mode refuses; a public-suffix name in private-CA mode refuses; real-domain mode on a host with no public record but a resolvable

ID	Control, location and test	Threats	Status
	zone starts; a zone NXDOMAIN refuses; a DNS timeout starts and logs; • the issuer check alerts when the served leaf does not match the mode; • a certificate for an out-of-constraint name issued under the private root is rejected by each supported browser.		
C-77	Staging credential isolation (ARCH §7.5, Principal, 2026-11-06): • **Its own keyring.** Staging's keyring has distinct `key_id`s, lives in its own secrets directory, and is mounted into staging `worker` only. • **Sandbox secret only.** Staging holds Plaid's sandbox secret, never the production secret. The `client_id` is shared across Plaid environments, and the secret is not. • **Startup checks.** Staging refuses to start unless `PLAID_ENV=sandbox`. Prod refuses to start with sandbox. • **SimpleFIN on staging** accepts only access URLs whose host is the demo bridge (exact host unmeasured; pinned at POLY-77). • **No real data.** No real credential may ever be linked on staging. **Lives at:** config loader; `packages/import-plaid`, `packages/import-simplefin`; INFRA **Tested by:** Config-loader unit tests for both refusals; an adapter unit test rejecting a non-demo SimpleFIN host under `APP_ENV=staging`; a provisioning check that the staging and prod secret directories share no file.	T-76, T-65	POLY-77 (the first real adapter on staging)
C-80–83	Supply chain (§8). **Lives at:** CI **Tested by:** §8	T-63, T-64, T-66	POLY-M1

8. CI/CD Security

- **C-80 Dependencies.**
  - Installs use `pnpm install --frozen-lockfile`.
  - pnpm lifecycle scripts are blocked, with an explicit `onlyBuiltDependencies` allowlist.
  - `minimumReleaseAge` is at least 3 days in both pnpm and Renovate, so a freshly compromised release has time to be yanked before we pick it up.
  - Renovate never auto-merges major versions.
  - `pnpm audit --audit-level high` gates the build.
- **C-81 Workflow hardening.**
  - Actions are pinned by commit SHA.
  - Workflows default to `permissions: contents: read`, and widen per job only.
  - No `pull_request_target` that runs untrusted code.
  - No secrets are exposed to fork PRs.
- **C-82 Scanning.**
  - gitleaks runs in CI and as a pre-commit hook.
  - CodeQL (JS/TS) runs on PRs.
  - Trivy scans images and fails the build on fixable critical or high findings and on embedded secrets.
  - Each release carries a CycloneDX SBOM.
  - Each release image gets an `actions/attest-build-provenance` attestation.
  - No DAST in v1 (R-13).
- **C-83 Deploy path. Pull-based: adopted in ARCH §8.1 @ 6b2f11b.** A systemd timer on the box pulls the image by digest and verifies its build attestation before migrating. The attestation must be pinned to this repository and the release workflow, e.g. `gh attestation verify --repo richmosko/polycarpic --signer-workflow …`. A prod tag is accepted only if its signature verifies against the allowed signers file. The box holds a read-only token scoped to this repository with an expiry, or no token at all if the image package is public. The superseded push-over-SSH conditions are kept for reference:
  - the key is prod-only;
  - `authorized_keys` locks it to a `command=` forced script, with `no-pty` and `no-port-forwarding`;
  - the key lives in a GitHub environment with required reviewers;
  - staging uses a separate key.

- **Signing.** `/merge-pr` signs release tags (SSH signing) before the first prod deploy, because a tag triggers deployment. Per-commit signing is not required (R-10).

9. Logging, Monitoring & Detection

- **Logged:** request ID, user UUID, entity UUID, route, action, outcome, latency, job name and run ID.
- **Never logged:** T1 or T2 values, request or response bodies, memo or payee text, amounts, emails, cookies, `Authorization`, SQL parameters. The C-33 canary test enforces this.
- **Storage and retention.** JSON logs stay on the box, rotated, for 30 days. Postgres logging is set per C-75. An optional OTel export must pass through the same serializers.
- **Operator alerts**, sent by email through Resend:
  - more than 20 sign-in failures in 10 minutes;
  - MFA disabled, or a new passkey added;
  - a membership grant;
  - a provider link or unlink;
  - an export or a purge;
  - 3 failures of a revoke job;
  - a connection going `principal_invalid`;
  - a backup failure, or no successful backup within the RPO;
  - an RLS battery failure on deploy (this also blocks the deploy);
  - more than 50 authz denials or empty scopes per user in 10 minutes.

  The thresholds are provisional and unmeasured; tune them after POLY-M1. *Revised:* 8963efb accepted no alerting at all (its R-2). Email on these events costs almost nothing.
- **User-facing notices.** Users get an email for a sign-in from a new device, a password or MFA change (including a first factor enrolled), and being granted or removed from an entity.
- **Audit.** `audit_event` (C-53) survives purge. The journal actor columns (C-23) are the ledger's own audit trail. C-24, the per-entity hash chain, makes ledger edits detectable. A weekly verifier checks it, and every export carries the chain head.

10. Incident Response

- **On call:** each instance's operator. For the Principal's instance, that is the Principal. Alerts arrive by email (§9).
- **Severity:**
  - **SEV1**: suspected T1 or T2 exposure, or keyring exposure; a confirmed cross-tenant read or write; code execution on the box; ledger tampering detected.
  - **SEV2**: T3 exposure limited to one user; no backup within the RPO; a revoke that cannot complete; a critical CVE with a known exploit in a deployed dependency.
  - **SEV3**: a vulnerability with no evidence of exploitation; a threshold breach that turns out benign.
- **Runbooks.** These cover the security content only; exact commands live in the INFRA runbook.
  - **RB-1: keyring or worker compromise (SEV1).**
    1. Stop `worker`.
    2. Revoke every token at its provider: Plaid `/item/remove`, SimpleFIN bridge revocation, broker OAuth revoke. If the database and the keyring may both have been exposed, treat every token as burned.
    3. Generate a new keyring.
    4. Relink every connection.
    5. Rotate the provider API secrets.
    6. Review `audit_event` and the logs for credential opens that happened outside import runs.
  - **RB-2: web or box compromise (SEV1).**
    1. Put Caddy into maintenance mode.
    2. Snapshot the volume for forensics.
    3. Rotate every operator secret, including the BetterAuth secret. This invalidates all sessions.
    4. If the worker or the keyring was reachable, run RB-1.
    5. Rebuild from a clean, attested image on a fresh host.
    6. Restore the last backup taken before the compromise, and verify it against the C-24 chain and against the head of the last export the owner holds off the box.
  - **RB-3: cross-tenant leak bug (SEV1).**
    1. Disable the affected route.
    2. Add the failing case to the RLS battery or the authz matrix *before* fixing it.

3. Identify affected users from request logs, by route and user ID.
4. Notify the affected members.

- **RB-4: lost keyring (SEV3).** Generate a new keyring. Every connection then shows "relink required". The ledger is unaffected.
- **RB-5: backup failure or restore (SEV2).** Follow INFRA. Restore to staging first, and run the balance and hash verifier before promoting.
- **RB-6: compromised dependency (SEV1 or SEV2).**
  1. Pin to the last known-good version.
  2. Rebuild and redeploy.
  3. Diff the SBOMs to check whether the bad version shipped in a release image.
  4. If it did, run RB-2.

- **Post-incident:** a blameless postmortem within 5 business days as a cairn issue comment or review file (location per `process/WORKFLOW.md`). Record decisions in process/DECISIONS.md. The regression test lands with the fix.

11. Open Risks (Accepted)

Each risk has a rationale, a compensating control, a re-evaluation date and an approver. The rows pending before 2026-11-07 were approved together that day (P3). Later rows show their own status. ARCH cites R-1 and R-4, and both keep their meaning; the other numbers are reassigned from 8963efb.

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
R-1	The operator's superuser, and root on the box, bypass RLS. They can also disable the append-only triggers (`ALTER TABLE … DISABLE TRIGGER` , `session_replication_role`) and read the keyring.	The operator owns the box, the volume and the keys. No database control can bind them (PRD §6). *Corrected:* 8963efb said the triggers still stop the superuser. They do not.	C-24 hash chain (POLY-M2, P4): silent edits become detectable, and the chain head goes into every export. Superuser access only through `docker compose exec` or an SSH tunnel. C-71.	On any decision to host for others	Rich Mosko approved the principle (PRD §6, 2026-09-27), and re-confirmed it on the corrected text (2026-11-07, P3)
R-2	RLS does not stop SQL injection or code execution in `poly_app` ,	Binding the GUC to a DB-verified session token would gain little:	C-44 (parameterized SQL, lint, CodeQL); C-30 (`web` cannot	2027-03-31	Rich Mosko (2026

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	because the app itself asserts the tenant GUC.	`poly_app` has to read the session table for auth anyway.	open credentials); C-10 (no DDL or TRUNCATE).		-11-07, P3)
R-3	On Vercel, both projects live in one Vercel account. Anyone with access to that account or team, or a leaked Vercel token with env-read scope, can read the `jobs` keyring. Vercel functions are also public HTTPS endpoints, so the `jobs` project is reachable from the internet. It is guarded only by `CRON_SECRET`, with no user input. *Narrowed by OQ-13:* the web/worker split is kept.	Vercel is the tertiary, last-resort target (Principal, 2026-11-06). The residual risk is administrative (account-level), not a web-process compromise.	Separate projects and env scopes (C-61). Vercel account MFA and least-privilege team membership. Rotate `CRON_SECRET` on suspicion. If the two-project split proves infeasible, Vercel may run only the fake or CSV adapters, never a real aggregator credential. Hobby-plan cron runs at most once a day (measured, ARCH §7.6 @ e3b4297). On Vercel, provider-side revocation after an unlink can therefore lag up to about 24 h, and the token stays live at the provider until then. The unlink UI must say so.	Before POLY-77 ships on Vercel	Rich Mosko (2026-11-07, P3)
R-4	Staging shares the prod kernel and auto-deploys unreleased code from	It saves a second box. Staging's data	C-70 (separate compose networks, non-root, read-only	ARCH §7.2 @ 6b2f11	Rich Mosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	`main` (TB-10). *Updated in v1.1:* staging now holds its own keyring and Plaid Sandbox / SimpleFIN demo tokens (Principal, 2026-11-06).	and tokens reach no real bank, so what is at risk is a pivot into prod, not staging's own data.	containers, no Docker socket); C-74 (distinct secrets); C-77 (staging's own keyring, sandbox secret only, refuses production endpoints); staging sits behind Caddy basic auth or an IP allowlist (ARCH §7.1).	b keeps staging co-located. Re-evaluate if staging ever touches non-fake data, or the app is offered at larger scale.	(2026-11-07, P3)
R-5	Purged entity data persists in backups until the retention window rolls.	Rewriting encrypted backups is disproportionate.	The purge warning says so. INFRA bounds retention.	INFRA v1	Rich Mosko (2026-11-07, P3)

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
R-6	The multi-tenant design leaves a hosted offering possible. Reversing the non-goal would bring in GLBA, state privacy law and possibly PCI, and would make R-1, R-2 and R-13 unacceptable.	Hosting for others is a standing non-goal (PRD §3).	This row makes any reversal a deliberate, costed decision that re-opens §1.	Only on a deliberate decision to host	Rich Mosko (PRD §3, 2026-09-27)
R-7	Viewers are not required to use MFA, nor is the owner of a user's own Person while it is unshared (OQ-34). Owners and managers elsewhere are (P1).	Viewers cannot mutate data, link credentials or grant access. The unenrolled own-Person owner can edit only their own unshared books; exporting, purging, sharing, granting and linking a bank all force enrolment first.	C-04 rate limiting; onboarding leads with OAuth; new-device email (§9).	GA	Rich Mosko (2026-11-07, P1; 2026-11-08, OQ-34)
R-8	Losing the keyring makes sealed credentials unrecoverable, and rotation is a manual operator step.	Automating rotation would mean storing the new key somewhere automation can reach. Unrecoverable is the right failure mode here,	The rotate command is idempotent and resumable (C-35); the runbook states the failure mode plainly; the ledger is not encrypted under this key.	Annually	Rich Mosko (2026-11-07, P3)

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
		because recovery is simply relinking.			
R-9	User-defined entity types are readable across tenants, by anyone with a membership on an entity of that type (kickoff §2.4).	A viewer needs the type label to render the entity.	A narrow policy plus a `security_invoker` view expose the label and tax defaults only, never the creator ID (C-14). Users are told that custom type names are shared.	If types ever carry more than a label	Rich Mosko (2026-11-07, P3)
R-10	Commits are unsigned.	With per-agent worktree identities, commit signatures prove little. Branch protection already prevents unreviewed merges.	Branch protection (admins included, no force-push, required `cairn` check). Release tags are signed (§8).	First prod deploy	Rich Mosko (2026-11-07, P3)
R-11	Revoking a grant does not recall data a client already fetched: an open browser tab now, the native cache later.	This is inherent to any client. The next request is denied.	The native milestone must purge its cache on 401, 403 or an empty scope. SecEng reviews that milestone.	Native milestone	Rich Mosko (2026-11-07, P3)
R-12	*Confirmed (measured, better-auth 1.7.18, 2026-11-08).* BetterAuth stores	A DB read already exposes T3. The auth	Short sessions (C-02); step-up for sensitive actions	POLY-62	Rich Mosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	the session token in the clear in `session.token`; the stored row matched the cookie token. It has no hashing option. A hook that rewrites the token breaks the lookup, because `findSession` matches on the raw token. The browser cookie is `token.HMAC(auth secret)`. A DB read alone cannot forge one. A DB read plus the auth secret can, and so can a DB read alone if the `bearer` plugin runs without `requireSignature`.	secret lives in the host secrets directory, not in the database (§6.1).	(C-03); the `bearer` plugin off, or on only with `requireSignature: true` (§4.1).		(2026-11-07, P3, as conditional). Re-approved as measured: Rich Mosko (2026-11-08).
R-13	No DAST, WAF or penetration test.	The surface is small and sits behind Caddy, and no regime requires one. pgTAP asserts the high-value properties more precisely than a scanner would.	The RLS battery; SAST, dependency, secret and image scanning (§8); this threat model is reviewed at every milestone.	GA	Rich Mosko (2026-11-07, P3)
R-14	Cross-root read access (PT-3, tooling). `cairn serve` reads directories named in a committed `roots:` config (or `--repos`), which can be	The server is read-only and localhost-only, with no write path to a secondary root.	When more than one root loads, a startup stderr banner logs each resolved absolute root path. It is a	2027-02-21	Rich Mosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	outside the repo. A malicious `roots:` entry in a PR could point a reviewer's local board at a sibling directory, e.g. `../../.ssh`.	The `/api/board` payload carries only parsed cairn frontmatter, so there is no exfiltration channel. A directory without a `config.yml` is skipped before anything is read. Anyone who can commit a `roots:` entry already has a stronger vector, editing `cairn.py` or `board.js` directly, so a runtime path-jail would be theater.	human tripwire against a stale or mistyped `roots:` entry silently serving the wrong tracker. Re-evaluate if cairn ever gains a write path to non-primary roots.		
R-15	A parent-link or cap-table refusal ("would make an entity its own ancestor", "too deep") reveals that some link exists, possibly through entities the user cannot see (T-78).	The inserter must own both ends, so the bit is about paths between their own entities. The error names no entity, and naming nothing	Errors name no entity (C-25). Refusals are audited.	If links ever become insertable by a non-owner	Rich Mosko (2026-11-07, P3)

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
		would make the refusal unexplainable.			
R-16	A purge, including one inside an account deletion, runs immediately with no cancellable delay. A hijacked session that passes step-up can destroy an entity's books at once.	The Principal's ruling (OQ-23). A delay needs a background purge the design forbids, and would make the freshness guards stale.	Step-up within 5 minutes (C-03, with a second factor required for owners, P1). Typed entity name. A fresh export by the same user. Every member emailed immediately. Operator backups (C-73). R-5 bounds how long recovery is possible.	If a purge is ever reported as unwanted	Rich Mosko (2026-11-06, OQ-23)
R-19	Hostile or captured founder. Other owners cannot remove the founding owner in-app. A founder, or someone in control of the founder's session, can force out co-owners one by one, each after a decline or the 14-day expiry. *Replaces* the earlier deadlock text: the founder now breaks removal deadlocks (Principal, 2026-11-07).	Without a tie-breaker, a two-owner dispute blocks forever. The founder's extra power is limited to co-owner removal or demotion, and is never immediate.	Step-up with the second factor (C-03, P1). The decline or expiry wait. A notice to every owner, the target included, plus `change.forced` audit events (C-28). No extra founder power over invites, roles or the cap table. The operator CLI as the backstop.	On the first reported forced removal	Rich Mosko (2026-11-07)
R-18	Real-domain mode on the local box keeps a zone-	It is what keeps Google, Apple	The token is scoped to one zone and	On any	Rich Mosk

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	limited DNS-API token on the box (T-68). Whoever steals it can issue certificates for that zone, or publish records in it.	and GitHub sign-in on a box nothing can reach. The box is unreachable from the internet, so a stolen token gets no path in.	mounted into Caddy only (C-76). It is rotated at the provider on suspicion. CT-log monitoring of the zone is optional.	change to the local-box TLS mode	o (2026-11-07, P7)
R-20	Private-CA mode installs an operator root on each device (T-79). If browsers do not enforce its name constraint, a stolen root key could impersonate any site to those devices.	It is the supported fallback for an operator without a domain (P7).	A name-constrained root, its key offline, and only an intermediate on the box (C-76). The runbook says to remove the root from every device on decommission or on a move to a real domain.	At the first private-CA deploy, after measuring constraint enforcement. That measurement is a gate item for the first home-box deploy.	Rich Mosko (2026-11-07)
R-21	With TPM auto-unlock, a thief who takes the *whole*	The Principal's choice (P8). A	C-72: sealed to Secure Boot,	If the box	Rich Mosk

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	home box gets a machine that boots and unlocks its disk. The data is then protected only by OS login, SSH keys and app authentication, and remains exposed to an OS or service exploit, or to cold-boot and DMA attacks on the running machine.	passphrase at boot would leave the box down after every power cut. Theft of a whole box from a home is rare, and a pulled disk stays unreadable.	firmware password, no USB boot, no unauthenticated console. Unattended updates (C-76). Backups encrypted separately (C-73). On theft, run RB-2: rotate every secret and revoke every bank token.	moves out of the home, or holds data for others	o (2026-11-07, P8)
R-17	Aggregate inference. Roll-up figures shown to a parent's members, the economic view, and allocated income shown to a holder's owners can be combined to infer things. Dividing economic by gross gives the viewer's own look-through stake. Allocated income divided by stake gives a pass-through entity's tagged income totals, as a K-1 tells a partner. Observing "today" figures every day accumulates detail over time. *Widened by P2:* a parent member may request roll-up totals for any date range. For a small entity, one-day	Each figure is one the viewer is entitled to by the Principal's rulings. Inclusion in a parent is an explicit grant by the entity's owner, so the exposure is consented, provided the consent is honest.	Consent disclosure at inclusion and in the upward-consent notice (C-25). The fixed aggregate shapes (C-25, C-27). Own and member entities are unaffected.	With POLY-1.1, and on any cross-parent "my net worth" proposal (OQ-29)	Rich Mosko (2026-11-07, P2)

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	periods and day-to-day balance differences reveal individual amounts and their dates, though never payee, memo, account or counterparty.				

12. Rulings on ARCH Open Questions

12.1 Final rulings

OQ	Ruling	vs. 8963efb (as folded in ARCH @ eaa825e)	Detail
OQ-1	App-layer **asymmetric** sealing: HPKE with X25519, HKDF-SHA256 and AES-256-GCM. The AAD binds each credential to its connection, entity and provider. `web` seals; only `worker` holds the private keyring and can open. pgcrypto and KMS are rejected. Rotation is a re-seal job.	**Revised.** Under the old design, the symmetric master key was reachable from `web`.	§6.3–§6.5; C-30, C-31, C-35
OQ-2	ARCH §3.3 is confirmed, and additions (a)–(d) stand. This review adds (e)–(l): • (e) authz functions owned by `poly_authz`; • (f) context read as `nullif(…,'')`; • (g) TRUNCATE triggers; • (h) `search_path = ''` plus a definer-function allowlist; • (i) tenancy columns not updatable; • (j) views are `security_invoker`, with no materialized views; • (k) role hygiene; • (l) UUID keys. The permission matrix is the same for every entity type.	**Extended.** Export becomes owner-only, per PRD §5. Payload access stays manager+.	§4.2.1, §4.3; C-10 to C-20
OQ-3	Confirmed: the linking user is the job principal, `principal_invalid` exists, and there is no system clause. Additions: • entity-bound jobs run as the earliest owner; • a new `app.job_targets()` function; • reference-data policies are role-targeted; • Cash-in-Transit aging is computed on read.	**Extended.** The POLY-64 scope comment stays valid.	§4.4; C-60

OQ	Ruling	vs. 8963efb (as folded in ARCH @ eaa825e)	Detail
OQ-4	The purge function is SECURITY DEFINER, owned by a **NOLOGIN `poly_purge`** role. It uses role-targeted DELETE policies, and the append-only trigger makes an exception only when `current_user = 'poly_purge'`. ARCH's guard order is kept, plus two new guards: no writes since the export, and an assertion that zero rows remain.	**Revised.** A purge owned by `poly_owner` deletes zero rows under FORCE RLS (T-51).	§4.5; C-21, C-50
OQ-19	C-19 amended. The owner-on-both insert rule is kept. No `poly_app` or `poly_worker` policy reads `entity_rollup`; only `poly_rollup`'s role-targeted policies do, behind fixed aggregate shapes. The one-bit refusal disclosure is accepted (R-15).	**New in v1.1.** *Finding:* the cycle trigger must not be owned by `poly_rollup`, whose confined reads would miss a cycle outside the caller's descendants. Hence `poly_graph` (A-16).	§3.8; C-19, C-25
OQ-23	Recorded: no purge delay (Principal). The guards stand and every member is emailed immediately.	Q2 withdrawn	§4.5; R-16
OQ-24	Local-box posture C-76: router with no forward and UPnP off; IPv6 closed; a host nftables default-deny with a LAN-only allow; Caddy bound to the LAN address plus a `DOCKER-USER` rule; outbound limited to DNS, NTP and HTTPS; an outbound heartbeat. TLS: two modes, a real domain by default with a `.local` private-CA fallback (P7). Disk unlock: TPM (P8). Backups: nightly, to an append-only internet storage box (P9).	**New.**	C-76; T-67 to T-69
OQ-25	Confirmed, with the shapes in §3.8:	**New.** ARCH's "owners of a holder read their	§3.8, §4.6; C-25,

OQ	Ruling	vs. 8963efb (as folded in ARCH @ eaa825e)	Detail
	• **Junction read and `parents_of`.** Read with any membership on the parent, or owner on the child. `parents_of` returns names only, to the owner only. • **Upward consent.** The notice names the new top parent and what becomes visible. • **OQ-18 grain at depth.** Unchanged: depth widens who sees, never what. • **Holdings popover.** Ordinary RLS only. • **Cap table.** Candidates, holder read path and notification go to `poly_stake` and `poly_graph`. • **Economic view.** It discloses only the viewer's own look-through stake. • **Allocated income.** A `poly_stake` definer, totals only. • **Staging keyring.** C-77. • **Deletion purge invoker.** §4.6, C-52.	row" policy becomes the `app.my_stakes()` definer (A-18).	C-27, C-52, C-77
OQ-30	**Confirmed with changes.** `poly_journal`, NOLOGIN and NOBYPASSRLS, owns the numbering and chain triggers. Its grants are column-limited UPDATE on the counter, INSERT on the counter only from an `entity` AFTER INSERT trigger, INSERT-only on `journal_chain`, and a trigger-only wide SELECT. A BEFORE INSERT trigger on `journal_line` refuses a line once its entry is chained, which closes the `SET CONSTRAINTS … IMMEDIATE` early fire. It joins the hygiene set and the allowlist.	**New** (ARCH v1.4).	C-10, C-15, C-24; T-80; A-23
OQ-31	**Confirmed, with three conditions.** The revocation runs in the same transaction as the membership write that makes the second owner, under a lock on the entity row. The invitation's issue path is recorded and not updatable. It counts effective owners only.	**New** (ARCH v1.4).	§4.2.2; C-28; A-24

OQ	Ruling	vs. 8963efb (as folded in ARCH @ eaa825e)	Detail
OQ-32	**Confirmed, with additions.** The predicate tests the zone, not the host: `.local` or no public suffix means `private-ca`, offline; otherwise `real-domain`, with a publicly resolvable zone. A zone NXDOMAIN refuses start; a DNS timeout does not. The served leaf's issuer is checked against the mode by the provisioning and health checks.	**New** (ARCH v1.4).	C-76; A-26
OQ-34	**Confirmed (the Principal ruled option i, 2026-11-08).** The own-Person owner row is exempt from the second step while that Person is unshared: no other member, and not rolled up into a parent. Enrolment is forced before creating a non-Person entity, sharing the own Person (a first invitation or a roll-up edge), accepting a manager or owner role, and any step-up action. Viewing never triggers it. The held role is ARCH §3.1c's `pending_role`, which meets A-25.	**New** (ARCH v1.5).	§4.1; C-03; R-7; T-81; A-25, A-28
Email-anchored first enrolment (Principal, 2026-11-08)	**Ruled.** The first factor is enrolled only after the password and a confirmed code sent to the account email: single-use, 10 minutes, 5 tries, 3 sends per 15 minutes, bound to the session, current address only. An unenrolled account is deleted with the password plus that code, with no set-up. Once a factor exists, nothing changes. SecEng ruling: a completed first enrolment sets the session's step-up stamp, so the next Confirm it's you passes within 5 minutes. BetterAuth has no native step; the app wraps the enrolment endpoints (measured on 1.7.18, §4.1; A-31 to A-33).	**New** (closes the OQ-34 gap the Principal found).	§4.1, §4.6; C-03, C-52; T-82 to T-84; A-29, A-30

12.2 ARCH changes to re-fold (architect)

#	ARCH §	Change
A-	§3.3	`visible_entities()` and `has_role()` are owned by `poly_authz`, with a role-targeted SELECT policy on `membership`. Add `poly_authz` and

#	ARCH §	Change
1		`poly_purge` to the role list.
A-2	§3.4	The purge function is owned by `poly_purge`. Add the role-targeted DELETE policies and key the trigger exception on `current_user`. Add guards 4 and 8. Replace the text "a separate purge role … rejected".
A-3	§3.3	Use the `nullif` form of `app.current_user_id()`. Correct the claim "unset returns NULL". The pool-reuse test must run on a backend that has already been used.
A-4	§3.2	Add BEFORE TRUNCATE statement triggers. No login role holds TRUNCATE.
A-5	§3.2	The actor is forced by trigger, not merely defaulted. `actor_kind = user` requires `actor_id = app.current_user_id()`.
A-6	§3.1, §4.1	Tenant-visible primary keys are UUIDs. Replace `/entities/42`.
A-7	§3.1 `provider_connection`; §2.2 `packages/secrets`; §7 TB-4; §9	The sealed columns become `enc`, `ciphertext`, `key_id` and `alg_version`. `seal()` lives in `web`; `open()` lives only in `worker`. TB-4 becomes the web ↔ worker boundary. Revocation on unlink runs as a worker job. Operator secrets are files, not an env file.
A-8	§3.1 user; PRD §5 wording	On account deletion, the `user` row becomes a tombstone and entries are never UPDATEd. `actor_id` FKs are `ON DELETE NO ACTION`. Override BetterAuth's `deleteUser`.
A-9	§2.3 RawTxn	Adapters redact full account numbers, PANs and credential fields from the stored payload.

#	ARCH §	Change
A-10	§3.3	Views are `security_invoker`, with no materialized views over tenant data. Tenancy and actor columns are not updatable.
A-11	§3.1 entity	A parent link requires owner on both the child and the parent.
A-12	§3.3 system jobs	Entity-bound jobs run as the earliest owner. Add `app.job_targets()`. Cash-in-Transit aging is computed on read.
A-13	§6.2 / POLY-76	Add spike acceptance criterion (e): FORCE RLS, a role-targeted `pgPolicy` (`to:`), and a definer function owned by a non-owner NOLOGIN role must round-trip through `generate` and apply as `poly_migrator`.
A-14	§3.1, §5 (*gap*)	Attachments appear in PRD §5 export and purge but have no storage design. They need: a location; size and type limits; serving with `Content-Disposition: attachment` and `nosniff`; and inclusion in both purge and export.
A-15	§7 TB list	Add TB-11 (upload ↔ parser) and TB-12 (supply chain ↔ prod).
A-16	§3.3b, §3.1a, §3.3 roles. *Folded in ARCH v1.4: §3.3 roles, §3.3b.*	`app.check_rollup_edge()` and `app.check_equity_holding()` are owned by a new NOLOGIN, NOBYPASSRLS `poly_graph` role. It has graph-wide SELECT on `entity_rollup` and `equity_holding`, SELECT on owner `membership` rows and `entity.kind`, and INSERT on `notification`. Add it to the role list. Add a test: a cycle through an entity invisible to `poly_rollup`'s predicate is refused.

#	ARCH §	Change
A-17	§3.1 `entity_rollup` row. *Folded in ARCH v1.4: §3.1, §3.3 roles.*	`app.parents_of` is owned by `poly_rollup`, under a second role-targeted predicate: ancestors of entities the caller owns. It is on the allowlist.
A-18	§3.1a "Who reads", §4.5 "The read", §3.3 roles. *Folded in ARCH v1.4: §3.1a, §3.3 roles.*	Add a new NOLOGIN, NOBYPASSRLS `poly_stake` role. It owns `app.cap_table_candidates`, `app.allocated_income` and a new `app.my_stakes()`. That function replaces the holder-owner SELECT widening on `equity_*`. Reads are confined to `app.held_entities()` (and, for candidates, to members' own Persons). See C-27.
A-19	§3.4, §3.1a (*gap*). *Folded in ARCH v1.4: new §3.4b, §3.1a; the name shown ruled by the Principal (OQ-33).*	Purging an entity that is a *holder* on other entities' cap tables. Those `equity_holding` rows are append-only, and they are the other entity's history. SecEng proposes: • the purged entity's row becomes a tombstone (as the user row does), so those rows still resolve, to "former holder"; • its units allocate as outside owners from the purge date; • the owners of each affected entity are notified. Shape needed before POLY-1.0.

#	ARCH §	Change
A-20	§3.1 membership and invitation, §3.1a "Who writes", §3.3 roles. *Folded in ARCH v1.4: §3.1b, §3.3 roles.*	Co-owner approval (P5b, SECURITY §4.2.2): • `change_request` and `change_approval` tables; • direct writes to `membership`, `invitation` and `equity_*` gated on `app.owner_count = 1`; • a new NOLOGIN `poly_govern` role owning `app.apply_change`, with role-targeted writes on those tables; • approvers recomputed at apply time, with payload re-validation; • a 14-day expiry, run as a scheduled job that only flips the status; • re-evaluation on an owner's tombstone (§3.4a).
A-22	§3.1 entity, §3.4a, §3.3 roles. *Folded in ARCH v1.4: §3.1 entity, §3.1b, §3.4a.*	Founding owner (R-19 ruling, SECURITY §4.2.2): • `entity.founding_owner_user_id`, NOT NULL, set to the creator, with a trigger that it holds owner; • `app.force_change` and the founder transfer as `poly_govern` definers; • in §3.4a, a successor step for a founder of a multi-owner entity, and a tombstone re-check; • the transfer target becomes founder when a sole-owned entity is transferred.
A-21	§3.3b. *Folded in ARCH v1.4: §3.3b, §3.3 roles.*	The ancestor-invite notice (P5) is a second `poly_graph` trigger: AFTER INSERT on `invitation` (and on `membership` without an invitation) on a parent-kind entity. It walks the descendants and notifies their owners in the same transaction. Not on a role raise.
A-23	§3.2.7, §3.2.9, §3.3 roles (OQ-30). *Folded in ARCH v1.5: §3.1,*	`poly_journal`'s grants as in C-24: • role-targeted policies `TO poly_journal` on the counter, `journal_chain`, `journal_entry` and `journal_line`; • UPDATE on the counter limited to `last_number`, `head_number` and `head_hash`;

#	ARCH §	Change
	§3.2.7, §3.2.9, §3.3 roles.	• the counter row created by a `poly_journal` AFTER INSERT trigger on `entity`, with no INSERT grant to `poly_app`; • `journal_chain` append-only under C-21, with the purge exception; • a `poly_journal` BEFORE INSERT trigger on `journal_line` refusing a line whose entry is already chained; • tenant-RLS SELECT for `poly_app` and `poly_worker` on the chain and the head; • the C-24 tests added in v1.2.
A-24	§3.1 invitation, §3.1b (OQ-31). *Folded in ARCH v1.5: §3.1, §3.1b.*	Add `invitation.issued_via` (direct, or the applying `change_request`), not updatable. The 1 → 2 revocation runs in the membership write's transaction, after an entity-row lock taken before `owner_count` is read. Revoked tokens are refused at acceptance. It counts effective owners only. Add the C-28 tests added in v1.2.
A-25	§3.1 membership, §3.1b, §3.3 (held role). *Folded in ARCH v1.5: new §3.1c, a pending role on `membership`; every condition below is met.*	A role raised to manager or owner is not effective until the member enrols a second step (§4.1, C-03). ARCH chooses the mechanism, a pending role on `membership` or refusal at apply. Either must satisfy all of these: • the stored value every check reads stays at the previous role until enrolment, so a check that forgets the pending state fails closed; • `has_role`, the policies, `owner_count`, the approver set, the 1 → 2 transition and founder eligibility all see the previous role; • the only write that makes the role effective is a definer that requires an enrolled factor for that user, changes nothing else, and writes an `audit_event`; • an owner can cancel a pending raise as a lowering, with no approval; • on a multi-owner entity, a co-owner raise already approved is not re-approved at enrolment.
A-	§7.8 "Mode cross-	Align with C-76: • the suffix test is offline and decisive;

#	ARCH §	Change
26	check at startup" (OQ-32). *Folded in ARCH v1.5: §7.8.*	• the zone test is an SOA from a public resolver; • an NXDOMAIN refuses start, and a DNS timeout starts, logs and retries; • a public record for the box's own name is a warning; • the issuer is checked against the mode by the provisioning and health checks.
A-27	§7.8 "OAuth sign-in", `packages/auth` (providers). *Folded in ARCH v1.5: §2.2, §7.8.*	Add GitHub (Principal, 2026-11-08): "Google, Apple and GitHub work" in real-domain mode, and all three are disabled server-side in private-CA mode. GitHub's verified email comes only from `/user/emails` (primary and verified), with scopes `read:user` and `user:email` (C-05).
A-28	§3.1c "Exempt", "Keeping the second step", "The sign-in gate"; §2.4 `SecondStep` (OQ-34)	The own-Person exemption holds only while the Person is unshared: no other member *and* no `entity_rollup` edge with it as the child. Today whoever adds that edge is enrolled, because they own the parent. If they later give up the parent role, "no other member" alone would restore the exemption, so they could remove their last method while the parent's members still see the Person. Use the same predicate in the last-method trigger, the sign-in gate and `SecondStep.required`. Test: a user who rolled their Person into a Household and then left it as owner cannot remove their last method while the edge remains.
A-29	§3.1c "The sign-in gate", "Keeping the second step"; `packages`	The first-factor path is password, then the emailed code, then the factor (SecEng §4.1). Add an `enrolment_grant` (user, session, expiry, consumed) opened by a confirmed code. A BEFORE INSERT on the passkey table and a BEFORE UPDATE that marks TOTP verified refuse for a user whose `second_step_at` is null unless a live, unconsumed grant for that user and session exists, and consume it in the same transaction. The before-hook on `/two-factor/enable`, `/two-factor/verify-totp`, `/passkey/generate-register-options` and `/passkey/verify-registration` refuses without a

#	ARCH §	Change
	`/auth` (enrolment)	grant. The `otp` method is not configured. The session record carries `recent_auth_at`, which a completed first enrolment sets. Applies after `reset-mfa` too. *Amended v1.2.3 (measured):* the endpoint names above replace `/passkey/add-passkey`. The hook is a global `hooks.before`, because `databaseHooks` never run for `twoFactor` or `passkey`. The "by construction, not by GUC" paragraph of §3.1c is replaced by A-31 and A-32: the guard reads `app.session_id`, and the TOTP check moves to the `user` UPDATE.
A-30	§3.4a (account deletion)	A user with no second step confirms deletion with the password (or a fresh IdP round trip) plus the emailed code, with no factor set-up. That confirmation is scoped to the deletion plan of its session and covers the own Person's export inside the run; it never sets the general `recent_auth_at`.
A-31	§3.1c; §3.3 `withTenant`; `packages/auth` (database config)	BetterAuth's database is a routing pool. Inside `withTenant`, its `connect()` returns the request's transaction client, with `BEGIN`, `COMMIT` and `ROLLBACK` rewritten to savepoints; outside, it falls back to a plain pool. This puts every BetterAuth write in our transaction. Enrolment triggers can then read `app.session_id`, a refusal rolls back BetterAuth's earlier writes in the same request, and a write outside `withTenant` fails the guard closed. Measured on 1.7.18: the GUC was visible, the backend and transaction matched, rollback worked, and nested transactions became savepoints. Pin the BetterAuth version, and add an integration test that fails if a BetterAuth release changes how it calls the pool. ARCH also states the role BetterAuth's statements now run as (the `withTenant` login role) and that role's grants on BetterAuth's tables. The other option, keeping BetterAuth's own pool and the "one live grant per user" match, leaves the first TOTP verify half-applied when the guard refuses (A-32). SecEng recommends the routing pool.
A-32	§3.1c "Not completed until confirmed"; `enrolment_grant`; `recent_auth_at`	The first `/two-factor/verify-totp` sets `user.twoFactorEnabled`, replaces the session, and only then sets `twoFactor.verified` (measured). When that last UPDATE runs, the requesting session is deleted and a grant that cascades from it is already gone. So: (1) put the TOTP guard on BEFORE UPDATE of `user` when `twoFactorEnabled` goes false → true. That fires first, while the session and the grant still exist. It requires a live grant for the user and for `app.session_id`, and consumes it, recording `txid_current()`. (2) The `twoFactor.verified` guard passes only if the grant was consumed in the current transaction. (3) The grant is consumed before the session is deleted, so a cascade from the session no longer matters. (4) The new session is built

#	ARCH §	Change
		from the old one as loaded at the start of the request. A `recent_auth_at` written to the old row does not carry over, so stamp it on the new session's INSERT in the same transaction when a grant was consumed in it. The passkey INSERT guard is unchanged: no session is replaced on that path unless `createSession` is set.
A-33	`packages/auth` (BetterAuth options); §3.4a	Pinned options, each measured or code-read on 1.7.18: `account.encryptOAuthTokens: true`; a `databaseHooks.account` create/update before-hook that drops `idToken`; `twoFactor({ allowPasswordless: true })`, so OAuth-only users can enrol, with the before-hook requiring the fresh IdP round trip for them; `skipVerificationOnEnable` left unset (false); no `otpOptions`; `user.deleteUser.enabled` left false (the endpoint answers 404), with C-52's own route doing the deletion; the `bearer` plugin not installed, or installed with `requireSignature: true`; `session.freshAge` left at its default, since `requireRecentAuth()` is the real gate and the passkey plugin's freshness check is extra.

12.3 Asks on other owners

- **devops-engineer:**
  - §6.2: secrets as files, scoped per container; the keyring goes to `worker` only.
  - §6.6: an asymmetric backup key and an append-only backup store.
  - C-70/C-71: only Caddy publishes ports; the firewall lives in the cloud provider, not on the host.
  - C-75: Postgres logging settings.
  - C-83: a pull-based deploy is preferred.
  - Assert the connection-pooler mode on both targets.
  - *v1.1:* local-box provisioning and its checks (C-76); staging's secrets directory, keyring and Plaid sandbox secret (C-77).
- **ux-designer:** *closed in DESIGN v0.16 (2026-11-08): every item below is specified (§ Co-owner approval, § Notifications, § Account deletion). Open from v1.2: the consent text's first sentence is amended (C-25); the spec block and the Playwright check take the new text verbatim.* The co-owner approval screens (§4.2.2):
  - a request shown as pending on the members list and the cap table;

    - an approve / decline screen with step-up, showing exactly what changes;
    - the expiry and reminder copy;
    - the requester-confirm prompt after the owner set changes;
    - the invitee sees nothing until apply;
    - a founder marker in the member list;
    - the founder's force screen, shown after a decline or expiry, with step-up and plain copy that every owner, the target included, will be told;
    - the founder transfer and step-down screen;
    - the successor step in the account-deletion flow.
- **qa-engineer:**
    - Every row of §4.2.1 becomes a pgTAP assertion.
    - The battery adds: pool reuse on a dirty backend (C-12); WITH CHECK forgery; catalog coverage plus canary; the definer allowlist; `security_invoker`; TRUNCATE; purge negatives and the zero-rows-remain check (C-50).
    - The log canary (C-33) must land before the first real adapter.
    - *v1.1:* the C-19, C-25 and C-27 tests (graph triggers, roll-up shapes, holder reads), the §4.6 deletion-flow negatives (C-52), and the C-77 config refusals.
    - *v1.2:* the C-24 `poly_journal` tests, the C-28 1 → 2 revocation tests, the C-03 held-role tests, the C-05 per-provider verification tests, the C-76 startup matrix, and the C-25 consent-text exact match.
    - *v1.2.1:* the C-03 OQ-34 exemption tests, and the A-28 roll-up predicate test.
    - *v1.2.2:* the C-03 email-anchor tests (including the direct endpoint calls), the A-29 pgTAP guard, and the §4.6 unenrolled-deletion tests.
- **product-manager (v1.2):** PRD §5's consent requirement carries the amended first sentence (C-25): "members of the parent, and of any parent above it, see …".
- **product-manager (v1.2.1):** PRD §5's "optional for viewers" gains the own-Person exemption and the actions that end it (OQ-34, §4.1).
- **ux-designer (v1.2.1):** Set up a second step appears as an interstitial for an unenrolled user before Create entity, before the first invite on the own Person, and before Confirm it's you on any step-up action (ARCH §3.1c).
- **ux-designer (v1.2.2):** Set up a second step, for a first factor, gains an email-code step after the password and before the QR code or passkey prompt: "We sent a code to *address*", a resend control that respects the limit, and a plain message for an expired or used code. After set-up, the gated action continues with no second Confirm it's you. Account deletion for a user with no second step asks for the password and the emailed code, and shows no set-up.

- **product-manager (v1.2.2):** PRD §5's second-step requirement says the first factor is confirmed by a code sent to the account email, and that a user with no second step deletes their account with the password plus that code.
- **Principal (via team-lead), 2026-11-07 question set: all ruled.** Q2 was withdrawn after OQ-23. Q1 and Q3 to Q5 are restated as P1, P3, P4 and P6.
  - **P1. Ruled 2026-11-07:** a second factor is required for owners and managers, and optional for viewers. C-03 and R-7 are updated.
  - **OQ-34. Ruled 2026-11-08 (closed):** the own Person is exempt from P1 until the user's first real action (option i). §4.1, C-03 and R-7 are updated.
  - **P2. Ruled 2026-11-07:** any date range, with honest consent. The inclusion step and the upward-consent notice carry the disclosure (C-25), and the residual is folded into R-17 and approved. There is no period guard in ARCH.
  - **P3. Ruled 2026-11-07:** every pending §11 row is approved. R-1 is re-confirmed on its corrected text. R-3 and R-4 are approved on their updated text.
  - **P4. Ruled 2026-11-07:** the hash chain is built with the ledger in POLY-M2. The weekly verifier and the export head line are in scope (C-24).
  - **P5b. Ruled 2026-11-07:** on a multi-owner entity, every other owner approves invites, role raises, a co-owner's removal or demotion, and cap-table edits (§4.2.2, C-28, T-26). The edges are SecEng rulings in §4.2.2.
  - **R-19. Ruled 2026-11-07:** the target still votes, and a founding owner (the creator) may force a declined or expired co-owner removal, with step-up and a notice to all. The founder is removable only by stepping down or transferring, and names a successor on account deletion. R-19 now records the hostile-founder risk, approved.
  - **P5. Ruled 2026-11-07:** notify, never block. When an invitation to a parent goes live, every owner beneath is notified with the disclosure sentence and the pull-out link (C-25, T-73).
  - **P6. Ruled 2026-11-07:** there is no reopen switch. Invitation is the only way in, plus `polycarpic user invite <email>` (C-01, §4.1).
  - **P7. Ruled 2026-11-07:** the default is a real domain resolved only on the LAN, with a DNS-01 wildcard (R-18 approved). The supported fallback is `.local` with a name-constrained private CA, OAuth disabled, and email + password + second step (C-76). Plain-http sign-in is refused in both. R-20 was approved 2026-11-07, with the name-constraint check as a gate for the first home-box deploy.
  - **P8. Ruled 2026-11-07:** TPM-sealed automatic unlock on the home box (C-72, T-69, R-21).
  - **P9. Ruled 2026-11-07:** the home box backs up nightly to an append-only internet storage box. A rotated drive is optional (C-73).

Generated by the `seceng` agent via `/generate-secdoc`. Re-run when ARCH changes; log security decisions in process/DECISIONS.md.